



中国石油大学 (华东)
CHINA UNIVERSITY OF PETROLEUM

本科毕业设计（论文）

题 目：

基于 LayaAir 与 ECS 的 Roguelite 生存战斗游
戏

设计与实现

学生姓名：_____刘瀚文_____

学 号：_____2207020509_____

专 业：_____计算机科学与技术专业_____

班 级：_____软件工程 2205 班_____

指导教师：_____董玉坤_____

2026 年 6 月

学位论文原创性声明

本人所提交的学位论文《基于 LayaAir 与 ECS 的 Roguelite 生存战斗游戏设计与实现》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。本声明的法律后果由本人承担。

论文作者（签名）：_____ 指导教师确认（签名）：_____
年 月 日 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解中国石油大学（华东）有权保留并向国家有关部门或机构送交学位论文的复印件和磁盘，允许论文被查阅和借阅。本人授权中国石油大学（华东）可以将学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编学位论文。

保密的学位论文在_____年解密后适用本授权书。

论文作者（签名）：_____ 指导教师（签名）：_____
年 月 日 年 月 日

摘 要

随着 HTML5 与 WebGL 技术的成熟，浏览器端已能够承载中等复杂度的实时动作游戏。Roguelite 品类以单局随机性、局外元进度与构筑深度为核心乐趣，对架构可扩展性、数据驱动能力与运行时性能提出了较高要求。本文以毕业设计项目 Survivor And Fight 为对象，在 LayaAir 3.x 引擎与 TypeScript 技术栈上，设计并实现了一套面向 2D 俯视角生存战斗的完整游戏系统。

系统采用轻量级实体组件系统（ECS）作为玩法内核，将位置、速度、属性、技能、操控等数据与逻辑解耦为组件与系统，并通过分组调度与命名筛选器支撑玩家、怪物及可操控实体的批量更新。战斗层实现配表驱动的技能与效果执行链，支持子弹发射、穿透、分裂、连锁等组合效果；子弹与怪物通过对象池减少实例化开销；在实体规模达到阈值时，战斗快照可通过 Web Worker 异步重算，主线程负责采集与应用结果。元游戏层提供开始界面、选关与三大关爬塔式跑图，局内支持经验升级、随机奖励与技能装配界面。游戏引擎架构理解、物理模拟与模块化设计方面的研究为本文提供了理论依据^[19, 7, 5]。

本文从需求分析、总体架构、模块设计与实现、性能优化与测试验证等方面展开论述。测试表明：系统在目标平台上可稳定运行；五波压力测试中，同屏 1000 怪条件下对象池使 P95 帧时间由 76.50 ms 降至 48.60 ms；动态图集在千怪波次上提升平均 FPS 约 13.4%。Web Worker 已集成于架构，同窗定量消融留作后续工作。

关键词：LayaAir；实体组件系统；Roguelite；数据驱动；Web Worker；对象池；生存战斗

Abstract

With the maturity of HTML5 and WebGL, browsers can host real-time action games of moderate complexity. This thesis presents the design and implementation of Survivor And Fight, a 2D top-down survival combat game built on LayaAir 3.x and TypeScript. A lightweight ECS drives gameplay; combat is configuration-driven with object pooling and optional Web Worker offloading. A meta layer provides menus, level selection, and a three-act DAG run map. Testing shows stable operation; object pooling reduces P95 frame time under 1000 on-screen monsters, and texture atlasing improves average FPS on the heaviest wave. Quantitative ablation of Worker offloading under warmed-up conditions is left for future work.

Keywords: LayaAir, ECS, Roguelite, data-driven, Web Worker, object pooling, survivor-like combat

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	2
1.2.1 游戏引擎架构	2
1.2.2 实体组件系统与玩法架构	2
1.2.3 物理模拟、人群与性能	2
1.3 研究内容与论文结构	3
1.4 项目背景	3
1.5 主要创新点与特色	4
1.6 伦理与合规说明	4
1.7 相关游戏与竞品简要分析	5
1.8 术语与缩写预备说明	5
1.9 本章小结	5
第 2 章 相关技术与理论基础	6
2.1 引擎语言选择	6
2.1.1 H5 游戏引擎选型	6
2.1.2 TypeScript 与工程常量管理	7
2.1.3 主循环、预制体与引擎使用约束	7
2.2 项目玩法架构选择	7
2.2.1 Survivor-like 品类与核心循环	7
2.2.2 轻量实体组件系统 (ECS)	8
2.2.3 数据驱动与 JSON 配表	8
2.2.4 Roguelite 跑图与关卡结构	9
2.2.5 战斗性能与密集实体仿真	9
2.3 UI 架构选择	10
2.3.1 MVC 分层与全屏界面栈	10
2.3.2 战斗内 UI: HUD、暂停与技能装配	10
2.3.3 UI 架构选型的综合评价	11

2.4 本章小结	11
第 3 章 系统需求分析	12
3.1 可行性分析	12
3.1.1 技术可行性	12
3.1.2 经济与社会可行性	12
3.2 功能性需求	12
3.2.1 用例：局内战斗	12
3.2.2 用例：元游戏进入战斗	13
3.3 非功能性需求	13
3.4 需求优先级	13
3.5 用户角色与运行环境	14
3.5.1 用户角色	14
3.5.2 硬件与软件环境	14
3.6 数据需求	14
3.7 接口需求	14
3.8 约束与假设	15
3.9 功能需求详细说明	15
3.9.1 ECS 核心需求	15
3.9.2 战斗与技能需求	15
3.9.3 怪物与波次需求	15
3.9.4 元游戏与跑图需求	16
3.9.5 UI 与交互需求	16
3.9.6 配置与工具需求	16
3.10 非功能需求量化指标	16
3.11 需求变更管理	16
3.12 需求与测试用例映射	17
3.13 本章小结	17
第 4 章 系统总体设计	18
4.1 设计原则	18
4.2 系统分层架构	18
4.3 ECS 调度与战斗数据流	19

4.4 元游戏与配表模型	19
4.5 技术选型说明	19
4.6 类与模块划分	19
4.7 安全设计	20
4.8 可靠性设计	20
4.9 开发与部署流程	20
4.10 数据库与配表逻辑模型	21
4.11 系统顺序图（文字描述）	21
4.12 部署拓扑	21
4.13 容错与降级策略	21
4.14 可扩展性设计	21
4.15 安全与隐私设计扩展	22
4.16 总体设计评审结论	22
4.17 本章小结	22
第 5 章 核心模块详细设计与实现	23
5.1 ECS 核心模块	23
5.2 属性、移动与筛选器	23
5.3 技能与效果执行	23
5.4 子弹系统与碰撞	24
5.5 怪物系统	24
5.6 进度、元游戏与跑图	24
5.7 技能装配 UI	24
5.8 配置加载	24
5.9 系统初始化与主循环实现	24
5.9.1 入口与配置加载时序	24
5.9.2 SimpleEcsDemo 构造过程	25
5.9.3 init 阶段实体生成	25
5.10 战斗效果执行详细流程	25
5.11 子弹碰撞与连锁算法说明	26
5.12 怪物 AI 与波次协同	26
5.13 经验、升级与奖励闭环	26
5.14 元游戏界面与 UI 栈交互	27

5.15 技能装配 UI 实现细节	27
5.16 输入系统	27
5.17 重启与死亡流程	28
5.18 模块间依赖关系小结	28
5.19 典型场景实现剖析	28
5.19.1 场景一：玩家首次进入战斗	28
5.19.2 场景二：Tab 打开技能面板并更换装备	29
5.19.3 场景三：怪物密集时的 Worker 卸载	29
5.20 关键代码文件索引	29
5.21 功能模块与论文章节对照	30
5.22 设计模式应用总结	30
5.23 实现难点与解决方案	30
5.24 本章案例小结	31
5.25 本章小结	31
第 6 章 性能优化与工程实践	32
6.1 性能目标与瓶颈	32
6.2 对象池	32
6.3 Web Worker 卸载	32
6.4 其它优化	32
6.5 绿色计算与兼容性	32
6.6 内存与垃圾回收分析	33
6.7 帧预算分配建议	33
6.8 配表热更新与构建优化	33
6.9 可观测性：日志与调试	33
6.10 与同类 H5 游戏优化策略对比	34
6.11 低功耗与绿色计算补充	34
6.12 性能测试实验设计	34
6.12.1 实验目的	34
6.12.2 实验变量	34
6.12.3 实验步骤	34
6.12.4 预期结论方向	35
6.13 性能优化检查清单	35

6.14 与学院模板“绿色节能”要求的对应	35
6.15 工程度量与质量门禁	35
6.16 本章小结	35
第 7 章 系统测试与验证	36
7.1 测试策略	36
7.2 单元验证	36
7.3 功能测试	36
7.4 性能测试	37
7.4.1 测试环境	37
7.4.2 动态图集对比（历史两轮，前三波）	37
7.4.3 五波完整帧率（实测，2026-05-18）	38
7.4.4 对象池消融（第 4-5 波）	38
7.5 系统运行截图	39
7.6 跑图与 Worker 一致性	39
7.7 测试结论	41
7.8 测试环境配置	41
7.9 回归测试清单	41
7.10 用户体验测试	41
7.11 缺陷记录示例	42
7.12 验收标准与毕业设计对齐	42
7.13 单元测试详细说明	42
7.13.1 ECS 核心验证	42
7.13.2 公式解析器验证	42
7.13.3 属性修饰器验证	43
7.14 集成测试场景扩展	43
7.14.1 跑图可达性测试	43
7.14.2 装配同步测试	43
7.14.3 Worker 一致性测试	43
7.15 测试数据与截图要求	43
7.16 测试结论补充说明	44
7.17 本章小结	44

第 8 章 总结与展望	45
8.1 工作总结	45
8.2 不足与展望	45
8.3 社会、健康与文化影响	45
8.4 个人收获与工程能力体现	45
8.5 与模板要求的三项补充分析	46
8.5.1 社会、健康、安全、法律与文化影响	46
8.5.2 绿色节能与兼容性	46
8.5.3 项目管理应用心得	46
8.6 对后续学习者的建议	46
8.7 结束语	47
8.8 本章小结	47
致谢	48
附录	51
附录 A 名词术语及缩略词	52
附录 B 核心配表字段示例（节选）	53
附录 C 需求与测试用例映射（节选）	54
附录 D 缺陷记录（项目实测）	55

插图

4-1	系统分层架构示意	19
4-2	配表逻辑关系示意	21
7-1	游戏开始界面 (StartPanel)	39
7-2	三大关 DAG 跑图界面 (RunMapPanel)	40
7-3	局内战斗场面 (同屏多怪与弹幕)	40
7-4	Tab 暂停下的技能/Effect 装配面板	40
7-5	生存胜利后的三选一奖励面板	40
7-6	玩家死亡后的重启面板	41

表格

2-1	主流 H5/跨平台方案与本项目选型对比	6
2-2	Unity DOTS 与项目轻量 ECS 对比	8
2-3	UI 架构选型要点汇总	11
3-1	主要功能性需求一览	13
3-2	需求优先级 (MoSCoW)	13
3-3	运行环境建议配置	14
3-4	非功能需求量化指标 (建议验收值)	16
4-1	技术选型汇总	19
5-1	效果类型与执行器映射	23
5-2	核心源码文件索引	29
5-3	功能模块与论文章节对照	30
7-1	功能测试用例摘要	36
7-2	测试环境配置	37
7-3	测试关卡分波 FPS (接入动态图集, 前三波历史实测)	37
7-4	动态图集接入前后 FPS 对比 (前三波)	38
7-5	Level3 五波测试帧率 (单次运行)	38
7-6	性能对比实验 (对象池消融, 1000 怪 / 10s)	38
7-7	缺陷记录表示例 (撰写时替换为真实 Bug)	42
C-1	需求—测试映射 (节选)	54
D-1	缺陷记录表	55

第 1 章 绪论

1.1 研究背景与意义

移动互联网与浏览器平台的普及，使“打开即玩”的 H5 游戏成为独立游戏与教学实践的重要载体。相较于原生客户端，H5 游戏在分发成本、跨平台兼容与快速迭代方面具有优势：玩家无需安装包即可通过链接进入，开发者可利用 Web 技术栈统一维护逻辑与资源。与此同时，JavaScript 单线程事件循环、垃圾回收（GC）抖动以及浏览器渲染管线约束，也对实时战斗类游戏的架构设计提出了更高要求——任何一帧内逻辑、物理近似、碰撞与 UI 事件若叠加超限，都会直接表现为卡顿或掉帧。

Roguelite 品类以自动攻击、大量敌人、局内升级构筑为核心循环，对数据驱动能力、同屏实体规模与元游戏流程编排提出了系统化工程需求。与传统回合制 Roguelike 相比，Roguelite 更强调单局内的 Build 组合与节奏压缩，适合在毕业设计周期内交付“可玩闭环”；与纯 Arcade 射击相比，其随机奖励与路线选择又要求架构能够支撑频繁调参与内容扩展。Survivor-like 子类进一步将操作简化为走位与构筑决策，使技术难点集中在高密度实体模拟与技能效果组合，而非复杂输入映射。

Survivor And Fight 项目目标是在 LayaAir 3.x 上实现 2D 俯视角生存战斗游戏原型，覆盖主菜单、选关、爬塔式跑图、局内战斗、升级奖励与技能装配等完整闭环，并探索 ECS 架构、配表驱动技能、对象池与 Web Worker 等性能手段。游戏引擎是商业游戏开发的基础工具，其架构复杂、子系统耦合度高，开发者常面临可维护性与演化性问题^[19, 7, 2]。Ullmann 等提出的 SyDRA 方法表明，通过恢复子系统依赖关系可辅助架构理解与变更影响分析^[19]；Gregory 对现代引擎渲染、资源、场景图与脚本分层的论述，为本文“表现层—逻辑层—基础设施层”划分提供了参照^[7]。

在 Unity 等引擎上，物理模拟已用于人群疏散等场景验证^[5, 3]，说明将重计算与交互渲染分离、采用模块化设计的价值。Cantrell 等在 Unity 中实现基于物理压力的人群疏散模型，Petty 等强调仿真系统须经 V&V（验证与确认）^[15]；本项目虽非物理引擎级仿真，但怪物追逐、分离力与密集碰撞同样属于“多代理体局部相互作用”问题，可借鉴其分阶段验证与性能—真实性折中思路。本课题在 H5 环境下实践类似思想，具有工程与教学双重意义：工程上形成可运行原型与性能数据；教学上贯通需求、设计、实现、测试的完整软件生命周期。

1.2 国内外研究现状

1.2.1 游戏引擎架构

SyDRA 方法通过对多款开源游戏引擎进行子系统依赖恢复,生成可比较的架构模型,帮助开发者完成架构理解与影响分析任务^[19]。Gregory 系统论述了现代游戏引擎的渲染、资源、场景图与脚本等子系统划分^[7]。Munro 等对 Quake 系列引擎的架构研究、Agrahari 对 Unreal 内部结构的分析,均为理解引擎模块边界提供了参考^[12, 1]。Goslin 介绍的 Panda3D 引擎体现了脚本与渲染协同的典型结构^[6]。Ullmann 等进一步可视化游戏引擎子系统耦合模式,指出过度耦合与目录嵌套是常见演化问题^[18]。

对 H5 引擎而言,除传统子系统外,还须关注资源异步加载、浏览器安全沙箱与主线程预算。LayaAir 等引擎在 2D 预制体、UI 与 TypeScript 脚本方面提供一体化工具链,使教学项目能在 IDE 内完成场景编辑与预览;但若将材质、贴图置于 `src` 而非 `assets`,运行时会出现 404,反映出“源码目录-资源发布目录”的部署约束。本文在总体设计中明确表现层(Laya 节点)与逻辑层(ECS)边界,避免 UI 直接改写战斗实体节点,以降低耦合度^[18, 4]。

1.2.2 实体组件系统与玩法架构

实体组件系统(ECS)将对象拆为实体标识、纯数据组件与无状态系统,有利于组合式扩展与逻辑批量处理。Unity DOTS 代表“面向数据”的极致路线;教学与中小型项目常采用 Map 存储组件、单线程顺序更新的轻量实现,在清晰度与性能间折中。Survivor-like 玩法强调波次刷怪与升级曲线;Roguelite 常采用 Slay the Spire 式 DAG 跑图组织关卡推进。

1.2.3 物理模拟、人群与性能

Reynolds 的 Boids 模型、Thalmann 与 Pelechano 的人群仿真工作,为大量代理体运动提供了经典范式^[16, 17, 14]。Cantrell 等在 Unity 中实现基于物理压力的人群疏散模型并完成验证^[5];Bott 等使用 Unreal 开发工具实现物理驱动的人群移动^[3]。McKenzie 等将人群行为建模与游戏技术结合用于仿真^[10]。上述研究对本项目怪物追逐、分离力与碰撞简化具有借鉴意义。H5 侧常见优化包括对象池、减少每帧分配、Web Worker 卸载重计算等。

1.3 研究内容与论文结构

本文主要研究内容包括：（1）基于 LayaAir 与 TypeScript 的总体架构；（2）轻量 ECS 核心与玩法系统；（3）配表驱动技能、子弹与碰撞；（4）元游戏流程与 DAG 跑图；（5）MVC 与 UI 栈；（6）对象池与 Worker 性能优化；（7）测试验证及社会影响分析。

全文共分八章：第 2 章从引擎与语言、玩法架构、UI 架构三方面介绍相关技术与选型依据；第 3 章给出需求分析；第 4 章描述总体设计；第 5 章详述模块实现；第 6 章讨论性能优化；第 7 章说明测试；第 8 章总结与展望。

1.4 项目背景

Survivor And Fight 是一款面向浏览器平台的 2D 俯视角 Roguelite 生存战斗游戏原型，采用 LayaAir 3.x 与 TypeScript 开发。项目目标是在可运行的工程基础上，形成覆盖主菜单与选关、爬塔式 **DAG** 跑图、局内自动战斗、升级奖励与 **Tab** 技能装配的完整玩法闭环，并验证轻量实体组件系统（ECS）、JSON 配表驱动技能、对象池与 Web Worker 等架构与性能手段在 H5 环境下的可行性。

从产业背景看，国产 H5 游戏与小游戏平台对“即点即玩”、低安装成本的作品需求持续增长。Survivor-like 品类以极简操作与高密度敌人见长，Roguelite 元游戏以单局随机与路线选择提升重玩价值；二者叠加可在有限美术与关卡人力下快速迭代内容，适合作为本科毕业设计周期内的可交付原型。相较商业大作，本项目的价值不在于内容体量，而在于可扩展的客户端架构与可复现的性能数据，为后续玩法、技能与关卡扩展预留接口。

从技术背景看，H5 实时战斗面临 JavaScript 单线程、GC 抖动与渲染预算三重约束。同屏数百怪物与大量弹幕时，若仍采用每帧 `instantiate/destroy` 预制体、在主线程完成全部追逐与碰撞，帧时间易出现尖峰。因此项目在技术选型上强调：逻辑与表现分离（ECS + Laya 节点）、参数外置（配表 + `defines`）、可降级优化（对象池、逻辑暂停、Worker 回退）。游戏引擎子系统耦合与演化问题在文献中已有系统讨论^[19, 7, 2]；人群与多代理体运动仿真研究则为怪物追逐、分离力与密集碰撞简化提供了理论参照^[5, 9, 16]。

工程上，仓库已具备可编译的 LayaAir 工程、`docs/config/` 下技能/子弹/角色等 JSON 配表及部分 2D 美术资源。论文工作聚焦于对上述实现的需求归纳、架构阐释、模块说明、测试验证与性能分析，使“可玩的 Demo”上升为可评阅、可答辩

的完整毕业设计成果。

1.5 主要创新点与特色

本项目仍具有以下工程特色：

- (1) 轻量 ECS 与引擎解耦：在 LayaAir 上实现可扩展的实体—组件—系统内核，玩法逻辑集中于 EcsWorld 调度，ViewComponent 仅负责节点绑定与同步，避免业务代码与引擎 API 深度缠绕。
- (2) 配表驱动的技能效果链：技能拆为 Skill 与多行 SkillEffect，支持穿透、分裂、连锁等修饰器与子弹效果按序组合；策划可在不改核心代码的前提下调整冷却、伤害与效果段，当前配表含 70 余条效果配置，支撑构筑深度演示。
- (3) 元游戏与局内战斗联动：三大关 DAG 跑图生成、节点类型（战斗/Boss/休息等）与 MetaFlowController 入口载荷联动，Boss 关可携带更长生存胜利时间等参数，体现 Roguelite 路线决策与实时战斗的结合。
- (4) 同屏规模下的性能手段：BulletPool、MonsterPool 抑制实例化尖峰；实体数达标时战斗快照经 Web Worker 计算追逐与分离，主线程保留子弹碰撞与渲染；实测表明对象池可显著降低 P95 帧时间，动态图集在千怪波次下提升平均 FPS。
- (5) 可验证的 UI 与战斗协同：全屏界面经 UIStackManager 路由；Tab 打开技能装配时 GameSession.paused 暂停战斗逻辑，SkillLoadoutSyncSystem 保证面板状态与自动施法一致，形成可手工回归的完整交互闭环。

上述特色均可在运行原型与第 7 章测试数据中直接验证，区别于仅停留在设计稿层面的游戏选题。

1.6 伦理与合规说明

游戏涉及虚拟战斗与生命值减少，不涉及真实暴力。项目未采集用户个人敏感信息。若部署至公网，须遵守网络游戏管理与未成年人保护相关规定，在说明中标注适龄提示与游戏时长建议^[15]。论文第 8 章对社会影响作了补充讨论，满足学院模板建议项。

1.7 相关游戏与竞品简要分析

吸血鬼幸存者（**Vampire Survivors**）：极简操作 + 自动攻击 + 海量敌人，证明 Survivor-like 品类的市场验证^[7]。本项目借鉴其战斗节奏与升级构筑，但通过 ECS 与 JSON 配表实现更透明的逻辑扩展，便于在论文中说明模块边界与数据流。

杀戮尖塔（**Slay the Spire**）：卡牌 Roguelike 与节点地图结合，强调路线决策与风险收益^[7]。本项目跑图采用类似的 DAG 结构，但战斗为实时俯视角而非回合制，技术难点落在帧率与碰撞而非回合状态机。

Noita：法术模块组合带来高自由度构筑^[7]。本项目 effect 链式修饰器（修饰器须位于子弹效果之前）吸收了“组合深度”思想，但未实现完整物理模拟与像素级交互，以控制毕业设计范围。

其他 **H5** 生存类作品：多数采用 Cocos/Laya 预制体配合传统面向对象层次结构。本项目以轻量 ECS、配表与 Worker 作为差异化工程路径，侧重架构可维护性与性能可测性，而非与商业产品在内容量上对标。

1.8 术语与缩写预备说明

全文首次出现英文缩写时给出中文释义，详见附录 A。引擎 API 与核心类名保留英文，便于与源码对照。

1.9 本章小结

本章阐述了项目背景、国内外相关研究、工程特色及论文结构，为后续章节奠定理论基础。

第 2 章 相关技术与理论基础

本章从引擎与语言、玩法与逻辑架构、界面架构三条主线，说明 Survivor And Fight 所依托的技术选型及其理论依据，为第三、四章的需求与设计提供共同语境。各节在综述文献与业界实践的基础上，给出本项目在毕业设计规模下的取舍理由，避免将实现细节堆砌为孤立名词列表。

2.1 引擎语言选择

2.1.1 H5 游戏引擎选型

面向浏览器分发的 2D 游戏，常见技术路线包括基于 Canvas/WebGL 的轻量渲染库（如 Phaser、PixiJS）以及一体化游戏引擎（如 Cocos Creator、LayaAir）。前者灵活度高，但场景管理、预制体工作流、UI 与资源打包往往需自行拼装；后者提供 IDE、预制体实例化、帧循环与输入封装，更适合在有限周期内交付可演示闭环^[6, 7]。

本项目选用 **LayaAir 3.x**，主要基于以下考虑：（1）课程与实验室技术栈已积累 Laya 工程经验，降低环境搭建成本；（2）3.x 对 TypeScript 的一等支持与本项目源码语言一致；（3）2D 预制体、Area2D 与 UI2 组件（GBox、GButton 等）可满足元菜单、跑图与战斗 HUD 的界面需求；（4）内置预览与发布至 bin/ 的流程便于性能测试与答辩演示。Gregory 指出，商业引擎的价值在于将渲染、资源、场景更新与脚本生命周期统一管理^[7]；LayaAir 在 H5 场景下承担了类似“运行时平台”的角色。

与 Unity、Unreal 等原生引擎相比，H5 方案的分发成本更低，但受浏览器单线程模型与 GC 行为约束更明显。Ullmann 等对游戏引擎子系统耦合的研究表明，无论引擎种类，清晰划分“引擎层—游戏逻辑层”均有利于后期维护^[18]。因此本项目将 Laya 节点仅作为表现层，核心玩法放在自研轻量 ECS 中更新，而非把所有逻辑写入 Laya.Script 回调。

表 2-1 主流 H5/跨平台方案与本项目选型对比

方案	优势	劣势	本项目
Phaser/PixiJS	轻量、社区活跃	工作流需自建	未采用
Cocos Creator	生态成熟、工具链完整	与现有课设栈不一致	未采用
LayaAir 3.x	IDE 一体、TS 原生、2D/UI2 完备	文档与社区小于头部引擎	采用

2.1.2 TypeScript 与工程常量管理

脚本语言选用 **TypeScript**，在 JavaScript 基础上提供静态类型与模块化，便于在实体、组件、系统数量增长时保持接口稳定。Briand 等关于面向对象设计可维护性的实验表明，清晰的模块边界与一致的命名有助于降低理解成本^[4]。本项目将可调战斗常量、UI 阈值、路径正则等静态配置集中于 `src/defines/`，经 `index.ts` 统一导出，避免在业务文件顶部散落魔法数；策划向的数值则外置 JSON 配表（见第 2.2 节），实现“工程常量”与“策划参数”的分工。

2.1.3 主循环、预制体与引擎使用约束

游戏主循环由 `Laya.timer.frameLoop` 驱动：每帧先执行 `EcsWorld.update(deltaTime)` 更新逻辑，再由引擎完成渲染与 UI 事件分发，符合 Gregory 所述“逻辑 tick 与绘制分离”的常见模式^[7, 20]。入口脚本 `Main.ts` 挂载在 `Area2D` 子节点而非 3D 根节点，避免纯 2D 战斗 Demo 误入 3D 渲染管线，属于针对本项目的引擎裁剪实践。

使用 Laya API 时须统一 `Laya.` 前缀（如 `Laya.Sprite`、`Laya.Handler.create`）。角色与子弹通过 `Laya.Prefab.instantiate` 生成；可加载资源（材质、贴图、预制体）须置于 `assets/` 发布目录，不可放在 `src/`，否则预览时会出现路径解析失败。2D 节点位移须通过 `transform.position = new Laya.Vector3(...)` 或 `translate` 完成，不可对 `position` getter 返回的副本调用 `setValue`，否则会出现逻辑坐标已变但画面不动的隐蔽错误——该问题在俯视角相机跟随与怪物追逐调试中曾多次出现，写入本文以供后续开发者规避。

2.2 项目玩法架构选择

2.2.1 Survivor-like 品类与核心循环

Survivor-like 游戏的核心循环可形式化为：移动躲避 → 自动输出伤害 → 击杀获取经验 → 升级强化 *Build* → 更高密度敌人。与传统射击游戏相比，玩家认知负荷从“瞄准射击”转向“走位与构筑决策”^[7]。因此，客户端架构必须同时满足：（1）玩家持续移动时，碰撞与伤害结算仍稳定、可预期；（2）数分钟内多次升级，奖励能即时写入属性或技能栏；（3）技能数值可通过配表迭代，而不反复修改核心战斗代码。

本项目在 `SimpleEcsDemo` 中默认创建玩家实体并由 `PlayerAutoCastSystem` 自动施法，保留 `ControlSystem` 处理走位，以降低操作门槛、突出构筑玩法。经验获取由 `ExperienceSystem` 完成，击杀奖励与等级加成通过 `defines` 中 `MONSTER_EXP_REWARD_BASE`、

MONSTER_EXP_REWARD_LEVEL_BONUS 等常量调节，属于可配置的“节奏设计”而非硬编码逻辑。

2.2.2 轻量实体组件系统（ECS）

实体组件系统（ECS）将游戏对象拆为实体标识（Entity）、纯数据组件（Component）与无状态系统（System）。其工程收益在于：新增怪物行为时往往只需增加 System 或扩展组件字段，而无需维护庞大的继承树；数据布局也有利于按类型批量处理^[7]。

Unity DOTS 路线采用 ECS + Job System + Burst，面向数万实体级并行^[7]。毕业设计团队规模与周期有限，若引入完整 DOTS 等价物，学习与 Laya 节点绑定的成本过高。本项目采用教学向轻量 ECS：EntityId 为数值类型；ComponentStore 按组件类型使用 Map 存储；EcsWorld.update 单线程顺序调用已注册系统，并按 input/logic/physics/render 分组排序。第 1 期目标为“结构清晰、支撑数百同屏实体”，而非追求极致数据导向布局。

表 2-2 Unity DOTS 与项目轻量 ECS 对比

维度	Unity DOTS	本项目 ECS
并行模型	Job System 多线程	单线程；追逐等片段可选 Web Worker
内存布局 与引擎绑定	Archetype / Chunk Entities Graphics	按组件类型的 Map ViewComponent 绑定 Laya 节点
适用规模	数万实体级	数百实体级
迭代与调试成本	高	中低

玩法层组件包括 PlayerTag/MonsterTag、Position/Velocity、Attribute（含可溯源 modifier）、Skill、SkillLoadoutState 等；系统包括 MovementSystem、AttributeSystem、SkillSystem、BulletSystem、MonsterChaseSystem 等。FilterRegistry 提供 Players、Monsters 等命名筛选，避免各 System 重复编写交集查询。怪物与玩家共享位移、属性、技能等组件，仅通过标签区分，体现组合优于继承的原则。

2.2.3 数据驱动与 JSON 配表

Roguelite 与动作游戏普遍采用数据驱动：将冷却、伤害、子弹参数、效果链等外置为表，由运行时加载^[7]。常见实现方式包括：（1）运行时加载 JSON 并反射填充数据类；（2）构建期生成 TypeScript 常量；（3）注册表声明表结构、运

运行时按路径加载。本项目采用第(3)种: `tables.registry.json` 声明表名与文件, `ConfigBootstrap.ensureGameConfigLoaded()` 在启动时加载, `initData` 将行挂载至 `Data.Character`、`Data.Skill` 等命名空间。

技能语义拆分为 `Skill` 与多行 `SkillEffect`: 每行对应子弹、穿透/分裂/连锁修饰或直伤之一, 由 `EffectExecutor` 按顺序解释——修饰器须位于后续 `bullet` 效果之前, 形成链式组合。伤害字符串(如 `"atk*1.5+10"`)由 `FormulaParser` 在白名单运算下解析, 禁止 `eval` 以防配表被篡改时产生代码注入风险。`Targeting` 支持自动索敌与简单指向输入, 满足自动施法与后续扩展。

与可视化脚本相比, JSON 配表便于 Git 管理与差异审查; 代价是部分错误推迟到集成阶段发现, 因此项目以 `isGameConfigReady` 与单元验证脚本(公式、属性修饰器)作为质量门禁。

2.2.4 Roguelite 跑图与关卡结构

除单局战斗外, 元游戏采用 **Slay the Spire** 式 DAG 跑图: 每局生成三个 Act, 每 Act 为多层有向无环图, 节点类型包括休息、普通战斗、Boss、宝藏等^[7]。地图生成一般包含分层、节点抽样、连边约束与可达性校验; 本项目 `RunMapGenerator` 实现固定行数网格、末行 Boss、中间随机分叉, 并通过 `validateActReachBoss` 保证 Boss 可达。失败时回退保底图, 避免进度卡死。

`SeededRng` 使相同种子可复现地图, 便于录制答辩视频与回归测试。`MetaFlowController` 在用户确认节点后触发 `onEnterCombat`, 将节点载荷(如 Boss 关更长生存时间)传入战斗场景, 实现元游戏与局内逻辑的松耦合。

2.2.5 战斗性能与密集实体仿真

Survivor-like 后期同屏怪物与弹幕数量陡增, H5 客户端热点包括: 每帧大量碰撞检测、预制体反复实例化触发 GC、主线程追逐与分离计算。业界常见对策包括对象池、纹理合批、逻辑暂停与将纯数值计算迁至 Web Worker^[7, 10]。

本项目 `BulletPool`、`MonsterPool` 按预制体路径分桶复用节点; `BulletHitTest` 使用距离平方比较减少开方; `GameSession.paused` 在技能面板打开时早退战斗 `System`。当实体数达到阈值时, `CombatDataPrepareSystem` 将坐标等快照经 `CombatDataBridge` 提交 Worker, 在共享的 `combatWorkerLogic.ts` 中计算追逐、分离与摆动, 主线程再写回速度; 子弹碰撞因依赖 Laya 节点与阵营判定, 仍保留在主线程。Cantrell 等在 Unity 中实现物理人群疏散时同样强调“重计算与交互渲染分离”^[5]; Helbing

等关于恐慌人群的研究则提示，密集场景须关注分离力与局部稳定性^[9]，与本项目 `MonsterChaseSystem` 中的分离项设计相呼应。

上述选型使玩法架构在“可扩展”与“可测”之间取得平衡：ECS 与配表支撑内容迭代，池化与 Worker 支撑同屏规模，跑图 DAG 支撑元游戏节奏。

2.3 UI 架构选择

2.3.1 MVC 分层与全屏界面栈

游戏 UI 常见模式包括 MVC、MVP、MVVM 等^[8]。Heijstek 等通过实验比较架构描述的图文形式，指出结构化图示有助于理解系统边界；本项目在实现上采用 MVC 变体：

- **Model:** 来自 ECS 组件（如 hp、装配状态）或独立模型类（如 `RunMapPanelModel`、`RunMapState`）；
- **View:** Laya 预制体与 UI2 节点绑定，负责显示与布局；
- **Controller:** 继承 `UiControllerBase`，处理生命周期、输入与路由，不直接操作战斗场景中的实体节点。

元游戏包含开始、选关、跑图等多个全屏界面。若允许多个面板同时 `visible`，易出现遮挡与事件穿透问题。本项目以 `UIStackManager` 维护路由栈：`push(routeId)` 入栈并显示对应 Controller，`pop()` 出栈恢复上一层，保证任意时刻仅一个全屏界面占据焦点。该模式与 Gregory 所述“将界面状态机与玩法逻辑解耦”一致^[7]。

2.3.2 战斗内 UI：HUD、暂停与技能装配

局内 UI 包括主 HUD（血条、经验）、升级奖励面板、死亡重启面板与 **Tab** 技能装配面板。`SkillSelectPanelController` 监听 **Tab**：打开时将 `GameSession.paused` 置为真，战斗 System 在 `update` 入口早退，怪物与子弹逻辑冻结，但 UI 仍接收指针事件——即输入层与战斗逻辑层分离，避免“面板打开仍被怪物攻击”的体验问题。

技能装配采用长按拖拽：`SkillDragService` 在按下超过阈值后创建顶层代理图标，规避 `GBox` 裁剪导致的命中错误；`SkillSlotHitTest` 统一坐标系判断落点属于装备栏或效果槽。关闭面板时，`SkillLoadoutSyncSystem` 将 `SkillLoadoutState` 原子写回 `Skill` 组件与 `PlayerAutoCastSystem`，防止半状态导致仍施放旧技能。图标

由配表 `iconPath` 与 `SkillIconBinder` 加载，资源目录约定文件名与 `id` 对应，减少冗余字段。

跑图界面 `RunMapPanelView` 根据 DAG 状态渲染节点与连线，`MapNodeIconUtil`、`LineLayoutUtil` 负责布局；玩家只能选择与当前节点相邻且已解锁的边，规则由模型层校验而非 View 硬编码。

2.3.3 UI 架构选型的综合评价

综上，UI 架构选型可归纳为表 2-3。其核心思想是：界面不拥有玩法真相，玩法状态以 ECS 与配表为准，UI 仅通过 Controller 与 `SyncSystem` 读写；全屏流程用栈管理，局内装配用暂停位切断战斗 tick。该划分降低循环依赖，也使第 7 章功能测试用例（Tab 暂停、装配同步、跑图可达）具有清晰验收口径。

表 2-3 UI 架构选型要点汇总

问题	选型	理由
全屏导航	<code>UIStackManager</code> 路由	避免多面板同显
局内构筑	栈	
局内构筑	Tab + 拖拽 + <code>SyncSystem</code>	暂停战斗、原子同步
数据绑定	预制体 View + Controller	便于美术替换与动画
与战斗边界	禁止 <code>System</code> 操作 UI 节点	单向依赖、利于测试

2.4 本章小结

本章围绕引擎语言、玩法架构、UI 架构三个维度，阐述了 LayaAir + TypeScript、轻量 ECS + 配表驱动 + Roguelite 跑图、MVC + UI 栈的技术路线及取舍依据，并修正了与毕业设计规模不匹配的过度工程化方案（如完整 DOTS）。后续第三、四章在此基础上展开需求分析与总体设计；第五、六章则对应具体实现与性能优化。全文引用游戏引擎架构、人群仿真与软件可维护性等领域文献^[19, 7, 5, 8]，以支撑选型论述的规范性。

第 3 章 系统需求分析

3.1 可行性分析

3.1.1 技术可行性

项目选用 LayaAir 3.x 与 TypeScript，已实现 ECS 核心、战斗 Demo、元菜单、跑图生成、技能装配 UI 等模块。引擎支持 2D 预制体、帧循环与键盘输入；现代 Chromium 内核浏览器支持 Web Worker 与 WebGL2。SyDRA 等研究表明，理解引擎子系统结构有助于降低维护成本^[19]；Cantrell 等验证了商业游戏引擎用于复杂物理仿真的可行性^[5]，说明在引擎之上叠加领域逻辑（人群/怪物运动）具有先例。

配表经 `tools/sync-config-to-bin.ps1` 发布至 `bin/config/`, `ConfigBootstrap.ensureGame` 在启动时双通道加载（`fetch` 与 `Laya.loader`），并通过 `isGameConfigReady` 检查行数。单元验证脚本覆盖 ECS、公式与属性修饰器，可在不启动完整 UI 的情况下验证核心契约。技术风险主要包括：同屏千级实体时的帧率、Worker 与主线程逻辑一致性、UI2 拖拽命中——均已通过性能实验与手工用例部分缓解。综上，在毕业设计周期内完成可运行原型并撰写配套论文，技术路线可行。

3.1.2 经济与社会可行性

作为毕业设计原型，主要成本为开发与测试人力。游戏为奇幻战斗题材，须控制暴力表现并提示合理游戏时长；跑图中的休息节点提供节奏缓冲。

3.2 功能性需求

主要功能需求如表 3-1 所示。

3.2.1 用例：局内战斗

参与者：玩家。前置条件：配表已加载，`SimpleEcsDemo.init()` 完成。主成功场景：（1）输入移动；（2）自动施法；（3）子弹碰撞、穿透/分裂/连锁；（4）击杀获经验并升级；（5）Tab 打开技能面板暂停并装配。扩展：玩家死亡显示重启面板。

表 3-1 主要功能性需求一览

模块	需求描述
ECS 核心 玩法实体	创建/销毁实体；按类型存取组件；系统分组更新 玩家/怪物标签；移动、视图同步；hp/maxHp 与 modifier
技能战斗 怪物系统 进度系统	多 effect 技能；索敌；冷却；子弹表驱动伤害与穿透 波次刷怪；追逐；接触伤害；离屏回收 经验、升级、随机奖励
元游戏 跑图	开始/选关；进入战斗；生存计时胜利 三大关 DAG；节点类型；种子可复现
技能装配 UI 配置	Tab 暂停；三装备栏；拖拽装配 JSON 表注册加载；缺失时默认值与日志

3.2.2 用例：元游戏进入战斗

用户在 `MetaFlowController` 选择关卡或跑图节点，触发 `onEnterCombat`，显示战斗场景、初始化 Demo、启动生存计时（Boss 关使用更长胜利时间）。

3.3 非功能性需求

性能：目标 60 FPS；同屏怪物受常量与波次控制；配合对象池与 Worker。可维护性：模块边界清晰；静态配置集中于 `defines`。兼容性：优先 Chromium；Worker 不可用时主线程回退^[5]。

3.4 需求优先级

表 3-2 需求优先级（MoSCoW）

级别	需求
Must	ECS 战斗、子弹碰撞、移动、刷怪、配表
Must	主菜单进战斗、生存胜利计时
Should	跑图、技能装配、升级奖励、对象池与 Worker
Could	法杖三槽完整 UI、全量 Effect 独立特效
Won't	联机、账号、支付（本期）

3.5 用户角色与运行环境

3.5.1 用户角色

系统仅设玩家单一角色，无管理员后台。玩家可：浏览主菜单；选择关卡或跑图节点；在战斗中移动、打开技能面板、选择升级奖励；死亡后重启。未来若扩展存档，仍不改变“玩家”唯一角色定义。

3.5.2 硬件与软件环境

表 3-3 运行环境建议配置

类别	要求
处理器	四核 2.0GHz 及以上
内存	8GB 及以上
显卡	支持 WebGL2
浏览器	Chrome / Edge 最新稳定版
开发环境	LayaAir IDE 3.x、Node.js（工具脚本）、TypeScript

3.6 数据需求

逻辑数据实体包括：实体 **Entity**、角色 **Character** 行、子弹 **Bullet** 行、技能 **Skill** 行、效果 **SkillEffect** 行、跑图节点 **RunNode**、会话 **GameSession**、装配 **SkillLoadoutState**。实体运行时状态存于 ECS 组件，配表行存于 Data 静态缓存，跑图图存于 **RunMapState**。

3.7 接口需求

1. 配置接口：`ensureGameConfigLoaded(): Promise<boolean>`，失败时返回 `false` 并日志告警；
2. 战斗入口：`MetaFlowController` 的 `onEnterCombat(payload?: CombatEnterPayload)`；
3. 输入接口：`ControlInputSource` 供 `ControlSystem` 查询方向；
4. UI 栈接口：`UIStackManager.push(routeId)`、`pop()`；
5. **Worker** 协议：`CombatWorkerComputeRequest/Response` 字段须版本兼容。

3.8 约束与假设

- 假设用户浏览器开启 JavaScript 与 WebGL;
- 假设开发阶段已执行配表同步脚本;
- 单机本地运行, 无强联网要求;
- 2D 俯视角, 不考虑 Z 轴复杂物理;
- 同屏实体规模在数百级, 非 MMO 级。

上述接口与约束与第 3 章 MoSCoW 优先级表一致, 可作为需求裁剪与答辩演示范围的依据。

3.9 功能需求详细说明

3.9.1 ECS 核心需求

系统须提供实体创建与销毁 API, 销毁时自动移除该实体全部组件。组件类型以 TypeScript 类为键, 同一实体不得重复挂载同类型组件。系统注册须指定分组与优先级, `update` 调用顺序确定且可预测。须提供按组件类型遍历与多组件交集查询能力, 供筛选器与系统使用^[7]。

3.9.2 战斗与技能需求

玩家实体须具备 `hp/maxHp`, 可通过 `modifier` 动态调整上限与当前值。技能由配表定义, 包含冷却、`effect` 列表。自动施法系统须支持至少三个装备栏位独立冷却。效果类型须覆盖子弹、穿透修饰、分裂修饰、连锁修饰、直伤。子弹须携带阵营信息, 碰撞时仅对敌对阵营生效。连锁搜索半径由工程常量定义, 避免无界遍历^[9, 7]。

3.9.3 怪物与波次需求

怪物自玩家周围环带生成, 保持最小间距, 避免与玩家重叠。怪物须追逐玩家并可造成接触伤害。远离玩家或死亡后应回收实体与显示对象。怪物可配置自动远程攻击。击杀须给予经验, 经验公式可含等级加成项^[7]。

3.9.4 元游戏与跑图需求

启动后须可进入开始界面。用户可选择关卡或进入跑图界面。跑图每局生成三个 Act，每 Act 为 DAG。节点类型至少包括休息、战斗、Boss、宝藏（命名以代码枚举为准）。用户只能沿已解锁边移动。选择战斗节点进入局内场景，Boss 节点携带特殊胜利计时参数^[7, 7]。

3.9.5 UI 与交互需求

全屏界面切换须经 UI 栈。技能装配面板由 Tab 切换，打开时战斗暂停。拖拽装配须区分技能与效果两类，不可跨类放置。长按阈值可配置。血条须随 hp 变化实时更新。升级时须弹出奖励选择界面。死亡后须提供重启入口^[8]。

3.9.6 配置与工具需求

配表须通过 registry 注册，启动时批量加载。静态常量不得散落在业务文件。资源路径须位于 `assets` 而非 `src`。开发工具须提供配表同步脚本与 PDF 模板提取脚本（本论文已提供 Node 版提取脚本）。

3.10 非功能需求量化指标

表 3-4 非功能需求量化指标（建议验收值）

指标	目标值	测量方法
战斗帧率	≥ 55 FPS（均值）	Chrome Performance
配表加载时间	$\leq 3s$ （本地）	控制台计时
技能表行数	≥ 13	<code>Data.Skill.GetAll()</code>
效果表行数	≥ 74 （含展示）	配表统计
Worker 回退	功能无差异	对比测试
论文字数	≥ 20000 汉字	字符统计脚本
参考文献	≥ 15 篇	BibTeX 条目

3.11 需求变更管理

需求变更应在设计文档与版本控制中留痕：新增功能先更新第 3 章需求描述与第 4 章设计说明，再修改实现，避免“代码先行、论文滞后”。毕业设计答辩前应冻结功能范围，仅接受文档修订与缺陷修复，防止范围蔓延导致论文与代码不一致^[11]。

3.12 需求与测试用例映射

需求 ID 与测试用例映射示例：R-ECS-01（实体销毁清理组件）→ 单元验证 `ecsCorePhase1.verify.ts`；R-SKILL-01（Tab 暂停）→ T-F-05；R-MAP-01（Boss 可达）→ 跑图生成后 `validateActReachBoss` 断言。完整矩阵可在终稿附录中以表格形式给出，增强测试章节可信度^[15]。

3.13 本章小结

本章论证了可行性，列出功能与非功能需求及典型用例，下一章给出总体设计。

第 4 章 系统总体设计

4.1 设计原则

系统总体设计遵循以下原则，并在后续模块实现中反复体现：

- (1) 数据与逻辑分离：组件仅存数据，系统执逻辑；可调参数尽量外置 JSON 配表，工程常量集中于 `src/defines/`^[7]。该原则使策划可在不改动 TypeScript 的前提下调整技能冷却、子弹速度等，也便于版本管理与 diff 审查。
- (2) 单一职责：`BulletSystem` 负责子弹运动与圆碰撞；`EffectExecutor` 负责将配表 `effect` 字段映射为运行时行为；`MetaFlowController` 仅协调元游戏流程，不直接修改 ECS 组件。职责清晰可降低修改时的意外副作用，符合 Briand 等关于可维护性准则的实验结论^[4]。
- (3) 可验证：核心模块提供单元验证脚本（公式解析、ECS 核心、属性修饰器），变更后可在 CI 或本地快速回归，呼应 Petty 对仿真与交互系统 V&V 的要求^[15]。
- (4) 渐进复杂度：第 1 期采用轻量 ECS 与单线程调度；对象池与 Web Worker 作为可选增强，仅在实体规模达标时启用，避免过早引入难以调试的并发模型。
- (5) 低耦合：UI 通过 Controller 与 SyncSystem 读写 ECS 状态，禁止 System 直接操作 UI 节点；战斗数值 Worker 与主线程共享 `combatWorkerLogic.ts`，避免双份逻辑漂移^[18, 10]。
- (6) 可降级：配表缺失、Worker 不可用、跑图生成失败时均有明确回退路径，保证演示与测试不中断（详见第 4 章容错设计）。

上述原则共同支撑“教学可完成、工程可扩展”的目标：在毕业设计周期内先交付闭环，再为法杖编程、空间划分碰撞等留出扩展点。

4.2 系统分层架构

`Main.ts` 初始化输入、构造 `SimpleEcsDemo`，按配置进入元菜单或战斗，在 `frameLoop` 中更新逻辑与生存计时。

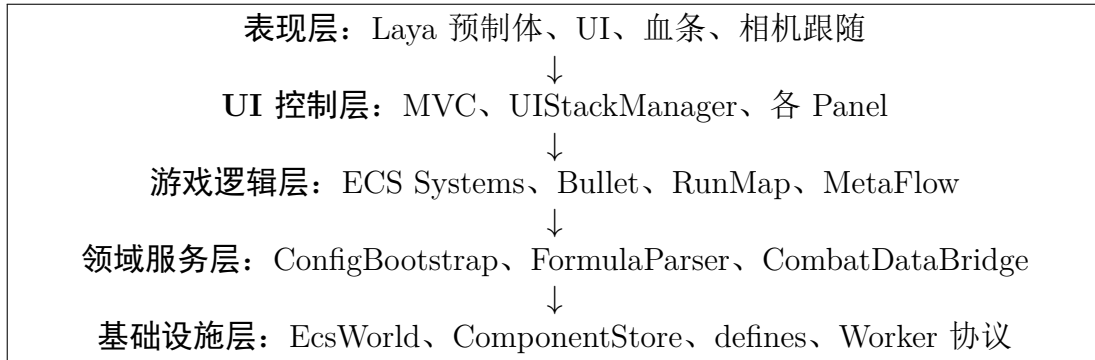


图 4-1 系统分层架构示意

4.3 ECS 调度与战斗数据流

系统分组: input (操控)、logic (属性、移动、技能、刷怪、经验等)、render (视图、血条、HUD)。GameSession.paused 控制战斗暂停。

战斗帧流程: CombatDataPrepareSystem 采集快照 → 可选 Worker 执行 processCombatFrame → 主线程应用追逐/分离结果 → BulletSystem 更新碰撞。该设计与 Cantrell 等将重计算置于引擎逻辑管线、渲染与交互分离的思路一致^[5, 3]。

4.4 元游戏与配表模型

MetaFlowController 协调界面与战斗入口; RunMapState 维护 DAG 进度。逻辑数据关系: Character、Skill 1:N SkillEffect, SkillEffect 可关联 Bullet; 跑图节点通过边构成 DAG。

4.5 技术选型说明

表 4-1 技术选型汇总

层次	选型	理由
引擎	LayaAir 3.x	教学栈一致、2D 预制体 workflow 成熟
语言	TypeScript	类型安全、与引擎脚本一致
架构	轻量 ECS + MVC UI	解耦、可扩展
配置	JSON + defines	策划可调 + 工程常量分离
并发	Web Worker (可选)	主线程减压
文档	需求/设计说明	与实现、测试可追溯

4.6 类与模块划分

核心类职责 (节选):

- EcsWorld: 实体组件系统门面;
- SimpleEcsDemo: 游戏场景组合根 (Composition Root);
- BulletSystem: 子弹模拟与碰撞;
- CombatDataBridge: Worker 桥接;
- MetaFlowController: 元游戏流程;
- RunMapGenerator: 跑图生成;
- UIStackManager: 界面路由;
- SkillSelectPanelController: 技能装配。

类图可在终稿中使用 PlantUML 绘制; 初稿以文字描述为主, 避免与快速迭代的代码签名不一致。

4.7 安全设计

H5 客户端安全重点包括: (1) 配表 JSON 不包含密钥; (2) 若未来接入 API, 须 HTTPS 与输入校验; (3) 避免在 `eval` 中执行不可信策划公式——`FormulaParser` 应仅支持白名单运算^[7]。当前单机原型攻击面较小, 但公式解析器仍是潜在风险点, 须在代码审查中禁止任意 JS 执行。

4.8 可靠性设计

Worker 失败时自动回退主线程; 配表加载失败时使用 `DEFAULT_*` 常量并 `missingConfigLogged` 去重日志; `RunMapGenerator` 生成失败使用 `generateFallback` 保底图, 避免卡死进度。重启流程通过 `restarting` 标志防止重入。

4.9 开发与部署流程

开发: Laya IDE 编辑场景与预制体 → TypeScript 编译 → 预览。配表修改后执行 `tools/sync-config-to-bin.ps1` 同步至运行目录。功能变更: 更新需求与设计说明 → 实现 → 单元验证与手工回归 → 修订论文对应章节。部署: 发布至 `bin/`, 由静态服务器托管, 无服务端渲染要求^[11]。

4.10 数据库与配表逻辑模型

虽无关系数据库，配表行仍构成逻辑 ER 关系：**Character** 1:1 关联玩家或怪物预制体；**Skill** 1:N **SkillEffect**；**SkillEffect** N:1 **Bullet**（当 effect 类型为 bullet）；**RunNode** N:N 通过边连接。图 4-2 以文字描述该关系。

```
Character(id, prefabPath, hp, maxHp)
Skill(id, cooldown, effectSlotCount)
SkillEffect(id, skillId, effect, damage, params)
Bullet(id, prefabPath, speed, penetration)
RunNode(id, type, payloadId) —edges→ RunNode
```

图 4-2 配表逻辑关系示意

4.11 系统顺序图（文字描述）

施法顺序图：PlayerAutoCast → CastPlan → EffectExecutor → BulletSystem → AttributeSystem（伤害）。升级顺序图：MonsterDeath → ExperienceSystem → UpgradeRewardSystem → RewardPanel → RewardApplyService → AttributeSystem。

4.12 部署拓扑

单机浏览器访问静态资源服务器；无后端集群。发布包包含 JS 编译产物、资源包、config JSON。CDN 部署时可缓存资源、短缓存 config 以便热更新^[7]。

4.13 容错与降级策略

1. 配表缺失：使用 DEFAULT_* 常量，记录一次日志；
2. Worker 失败：主线程 processCombatFrame；
3. 跑图生成失败：generateFallback；
4. 预制体加载失败：占位半径碰撞体，保证可继续调试；
5. UI 面板加载失败：跳过该路由并日志，不阻断战斗。

4.14 可扩展性设计

新增怪物行为：添加 System 或扩展 MonsterDef 组件字段。新增技能效果类型：扩展 EffectExecutor 分支。新增界面：注册 UI 栈路由与 Controller。新增跑图节

点类型：扩展 `RunNodeType` 与生成器权重。上述扩展均在不修改 `EcsWorld` 内核的前提下完成^[7]。

4.15 安全与隐私设计扩展

客户端不存储密码。若接入分析 SDK，须在隐私政策中披露。配表 JSON 禁止包含 API 密钥。用户输入仅用于游戏操控，不上传聊天记录。公式解析器禁止 `eval`，防止配表被恶意篡改时 RCE^[7, 15]。

4.16 总体设计评审结论

总体设计满足第 3 章所列 Must/Should 级需求，模块边界清晰，性能与可维护性有明确手段。风险在于法杖系统与部分 Effect 视觉未完全实现，须在论文与答辩中如实标注。下一章实现细节与本章设计一一对应，便于评阅。

4.17 本章小结

本章给出分层架构、ECS 调度、战斗数据流与元游戏设计，下一章详述模块实现。

第 5 章 核心模块详细设计与实现

本章结合 `src/` 实现,对核心模块作详细说明。`SimpleEcsDemo` 作为组合根 (Composition Root) 集中注册系统、池与 UI 控制器,是阅读源码的首选入口;表 5-2 (见本章扩展节) 给出模块与路径索引,便于评阅对照。

5.1 ECS 核心模块

`EntityManager` 使用自增 ID 与空闲列表复用实体。`ComponentStore` 按类型维护 `Map<EntityId, Component>`。`EcsWorld.update` 依次调用已注册系统,按 `input/logic/physics/re` 分组排序,与 Gregory 所述引擎主循环驱动子系统更新的模式相对应^[7]。

`ViewComponent`、`BodyUIComponent` 绑定 Laya 节点;`ViewSyncSystem` 同步坐标;`BloodBarSyncSystem` 用 `hp/maxHp` 更新 `ProgressBar`。

5.2 属性、移动与筛选器

`AttributeSystem` 支持 `modifier` 按来源增删,供 `Buff` 与升级奖励使用^[4]。`MovementSystem`、`ControlSystem` 处理位移与输入。`FilterRegistry` 提供 `Players`、`Monsters` 等命名筛选,减少重复查询。

5.3 技能与效果执行

`PlayerAutoCastSystem` 对三装备栏维护独立冷却与 `pendingCasts`。`EffectExecutor` 将配表 `effect` 映射为子弹、穿透/分裂/连锁修饰或直伤。修饰器须位于后续 `bullet effect` 之前,形成链式组合语义。

表 5-1 效果类型与执行器映射

effect 字段	行为
<code>bullet</code>	<code>BulletSystem.spawnBulletWithOptions</code>
<code>modifier_split</code>	设置分裂数量
<code>modifier_chain</code>	设置连锁与半径
<code>modifier_pierce</code>	增加穿透
<code>direct_damage</code>	公式直伤

5.4 子弹系统与碰撞

BulletSystem 从 BulletPool 取节点, 更新运动, 用 hitRadiusSq 做圆碰撞, 按阵营过滤后扣血并处理穿透。密集命中场景可参考人群仿真中的局部相互作用与压力模型思想^[9, 16], 本项目采用简化半径检测以降低开销。

5.5 怪物系统

MonsterWaveSpawnSystem 在玩家周围环带刷怪; MonsterChaseSystem 追逐并叠加分离力与随机摆动, Worker 可参与数值重算^[5]。MonsterContactDamageSystem 处理接触伤害; MonsterRecycleSystem 与 MonsterPool 回收离屏单位。

5.6 进度、元游戏与跑图

ExperienceSystem、UpgradeRewardSystem 与奖励面板构成成长闭环。MetaFlowController 协调菜单与 onEnterCombat。RunMapGenerator 生成三幕 DAG, 校验 Boss 可达, 失败时使用保底图。

5.7 技能装配 UI

SkillSelectPanelController 以 Tab 切换面板并设置 GameSession.paused。SkillDragService 长按拖拽, SkillLoadoutSyncSystem 写回战斗实体。UIStackManager 管理全屏路由, 生命周期由 UiControllerBase 约定。

5.8 配置加载

ConfigBootstrap 双通道加载 JSON 并 initData 填充 Data, 是数据驱动枢纽^[7, 11]。

5.9 系统初始化与主循环实现

5.9.1 入口与配置加载时序

Main.onStart 中依次完成: 创建 InputService、ControlInputAdapter、SimpleEcsDemo; 注册 frameLoop; 调用 loadConfigAndInitDemo。该异步函数 await ensureGameConfigLoaded(), 若 META_MENU_ENABLED 为真则 startMetaMenu, 否则直接 demo.init()。时序设计

保证配表优先于实体生成，避免 `Data.Character.GetByID` 返回空导致预制体路径缺失。

5.9.2 SimpleEcsDemo 构造过程

构造函数内完成世界初始化与系统注册，核心步骤包括：创建 `FilterRegistry` 并 `registerNamedFilters`；注册 `RESTART_PANEL_ROUTE_ID`；创建 `GameSession` 实体；实例化 `AttributeSystem`、`ExperienceSystem`、`BulletSystem`（注入 `BulletPool` 与 `CombatDataBridge`）、`CombatDataPrepareSystem`、`SkillSystem`、`MonsterWaveSpawnSystem`、`UpgradeRewardSystem`、`MainHudSystem`、`SkillSelectPanelController` 等。该集中注册点便于查阅系统拓扑，亦可在未来改为反射或配置文件驱动注册。

5.9.3 init 阶段实体生成

`init()` 在配表就绪后加载玩家预制体：读取 `Character` 行中 `prefabPath`、`bodyPrefabPath`、`hp/maxHp`；创建实体并挂载 `PlayerTag`、`Position`、`Velocity`、`Attribute`、`Skill`、`SkillLoadoutState`、`Control`、`Experience`、`ViewComponent` 等；默认写入 `createDefaultLoadoutState` 与自动施法技能 `id`。怪物由波次系统动态生成，而非 `init` 静态放置，以降低初始场景负担。

5.10 战斗效果执行详细流程

当 `PlayerAutoCastSystem` 决定施放技能时，流程如下：

1. 从 `SkillLoadoutState` 读取装备栏技能 `id` 列表，检查 `Skill.cooldownRemain`；
2. 调用 `CastPlan` 解析技能包含的 `effect` 行（按配表顺序）；
3. 对每个 `effect`，`EffectExecutor` 根据 `effect` 字段分发；
4. 若为 `modifier_*`，将分裂数、连锁数、穿透增量写入上下文，供下一条 `bullet effect` 消费；
5. 若为 `bullet`，合并上下文参数调用 `spawnBulletWithOptions`，设置 `ownerSide` 为玩家阵营；
6. 若为 `direct_damage`，`FormulaParser` 计算伤害并 `AttributeSystem` 扣血。

该“修饰器 + 子弹”顺序语义与 Noita 式法术链思想一致：修饰器本身不直接产生视觉效果，而是改变后续子弹行为^[7]。实现上须保证 effect 表内顺序可被策划理解，建议在工具链中提供顺序预览。

5.11 子弹碰撞与连锁算法说明

设子弹 b 与怪物 m 圆半径分别为 r_b, r_m ，中心距离平方 $d^2 = (x_b - x_m)^2 + (y_b - y_m)^2$ ，若 $d^2 \leq (r_b + r_m)^2$ 则判定命中^[9]。命中后：

- 对 m 应用 damage（可含公式）；
- penetration 减 1，为 0 则回收子弹；
- 若存在 chainCount，在 CHAIN_HIT_RADIUS 内选取下一未命中目标，递归扣血直至次数耗尽；
- 若存在 splitCount，在原位置扇形生成多颗衍生子弹（速度方向扰动）。

怪物子弹对玩家同理，仅阵营判断相反。该逻辑在 spec 中要求“玩家子弹仅对 MonsterTag 生效”，防止误伤。

5.12 怪物 AI 与波次协同

MonsterChaseSystem 每帧计算指向玩家的单位方向向量，乘以 MONSTER_CHASE_SPEED 写入速度或位移。密集人群时叠加分离力：当两怪距离小于 MONSTER_SEPARATION_DISTANCE，沿连线方向施加排斥，强度 MONSTER_SEPARATION_FORCE，降低模型重叠。随机摆动项使用 MONSTER_RANDOM_SWAY_DEGREE 与 MONSTER_RANDOM_SWAY_FREQ 增加轨迹不可预测性。

MonsterWaveSpawnSystem 根据当前存活数与上限 MONSTER_COUNT 决定刷怪，使用 pickMonsterIdForWave 从 MonsterCatalog 按波次权重选 id。MonsterRecycleSystem 在怪物远离玩家超过阈值后回收实体与节点，维持活跃集合规模。

5.13 经验、升级与奖励闭环

Experience 组件记录当前经验与等级。ExperienceSystem 在怪物死亡时读取 ExperienceReward，增加经验并判断是否升级。升级后 UpgradeState 标记待选奖励，UpgradeRewardSystem 暂停战斗或打开 RewardPanel（具体交互由 UI Controller

实现)。RewardPoolService 按权重抽取奖励条目，RewardApplyService 可能执行：增加属性 modifier、提升 hp 上限、解锁技能等。

该闭环将“生存时间”转化为“数值成长”，是 Survivor-like 品类留存关键^[7]。配表 rewardDefine.ts 中常量控制池子 id 与 UI 路由。

5.14 元游戏界面与 UI 栈交互

MetaMenuBootstrap 向 UIStackManager 注册 StartPanel、SelectLevelPanel、RunMapPanel 等路由。用户点击“开始”后 Controller 调用 MetaFlowController 方法切换栈顶界面。选关或跑图确认后触发 onEnterCombat 回调，Main 中隐藏菜单容器、显示战斗 Area2D、调用 demo.init() 并启动生存计时。

RunMapPanelView 根据 RunMapState 渲染节点图标与连线，MapNodeIconUtil、LineLayoutUtil 负责布局。玩家仅可选择与当前节点相邻且已解锁的边，保证 DAG 规则。

5.15 技能装配 UI 实现细节

SkillSelectPanelView 绑定 MainUIPanel.1h 预制体。初始化时 SkillIconBinder 根据配表加载图标纹理。SkillDragService 在指针按下超过 LONG_PRESS_MS 后进入拖拽模式，使用顶层代理图标跟随指针，避免被 GBox 裁剪。SkillSlotHitTest 判断落点属于装备栏、Effect 槽或背包区，SkillDragService 仅允许同类 DragKind 放置。

关闭面板时 SkillLoadoutSyncSystem 将 SkillLoadoutState 同步至 Skill 与 PlayerAutoCastSystem 的 pending 列表，并调用 getCombatEffectIds 刷新可战斗 effect 集合。该同步必须是原子性的，防止半状态导致施法失败。

5.16 输入系统

InputService 封装键盘/指针事件，ControlInputAdapter 转换为 ControlSystem 可读的方向向量。输入层与 UI 层解耦：Tab 打开面板时，战斗 Control 不更新，但 UI 仍可接收指针事件。该分离符合 MVC 职责划分^[8]。

5.17 重启与死亡流程

PlayerDeathSystem 监测 $\text{Attribute.hp} \leq 0$, 触发死亡标志。RestartPanelSystem 通过 UI 栈显示重启面板, 玩家确认后 SimpleEcsDemo 执行 re-init: 清理实体、重置池、重新生成玩家与波次。须防止重复 re-init 导致监听器泄漏, restarting 标志用于重入保护。

5.18 模块间依赖关系小结

依赖关系遵循单向原则: UI $\rightarrow$ ECS 组件读写; Systems $\rightarrow$ 配表/Data; BulletSystem $\rightarrow$ BulletPool/CombatBridge; MetaFlow $\rightarrow$ Main 回调。禁止 Systems 直接操作 UI 节点, 须通过 Controller 或专门 SyncSystem。该约束降低循环依赖, 利于单元测试^[15]。

5.19 典型场景实现剖析

5.19.1 场景一: 玩家首次进入战斗

该场景串联配置、元游戏与 ECS 初始化, 是答辩演示的主路径。步骤分解如下:

步骤 1: 应用启动。Main.onStart 注册帧循环, 调用 loadConfigAndInitDemo。ensureGameConfigLoaded 遍历 configBases() 路径, 尝试 fetch 与 Laya.loader 加载 registry 与各表。若失败, isGameConfigReady 为 false, 技能面板与自动施法可能退化。

步骤 2: 元菜单。META_MENU_ENABLED 为真时, MetaMenuBootstrap 初始化 UIStackManager, push 开始面板。MetaFlowController 监听用户操作。战斗容器 Area2D 暂隐藏, 避免空场景渲染。

步骤 3: 进入战斗。用户选关后触发 onEnterCombat。容器可见, demo.init() 创建玩家、注册系统更新。生存计时器 CombatSurvivalTimer 启动, Boss 关使用更长胜利时间常量。

步骤 4: 首帧更新。ControlSystem 读取输入; PlayerAutoCastSystem 根据默认装配施放; MonsterWaveSpawnSystem 开始刷怪。血条 BloodBarSyncSystem 将 hp 映射至 ProgressBar。

该场景验证“配置—流程—ECS”贯通, 对应测试用例 T-F-01、T-F-09。

5.19.2 场景二：Tab 打开技能面板并更换装备

用户按 Tab, `SkillSelectPanelController` 将 `GameSession.paused` 置 `true`。战斗 System 早退, 怪物与子弹停止逻辑更新 (或仅冻结施法, 取决于具体 System 实现, 以代码为准)。面板显示三装备栏与 Effect 槽。

用户长按技能图标 300ms 后拖拽至另一槽, `SkillDragService` 更新 `SkillLoadoutState`。`SkillLoadoutSyncSystem` 写回 Skill 组件与自动施法列表。关闭 Tab 后恢复战斗, 新技能在下一冷却周期生效。

该场景验证 UI 与 ECS 状态同步, 对应 T-F-05、T-F-06。若同步遗漏, 可能出现“面板已换技能但仍在放旧招”的缺陷, 属于高优先级回归项。

5.19.3 场景三：怪物密集时的 Worker 卸载

当场上怪物数量达到 `COMBAT_WORKER_MIN_ENTITY_COUNT`, `CombatDataPrepareSystem` 打包坐标快照, `CombatDataBridge` 发送至 Worker。 `processCombatFrame` 计算追逐、分离、摆动, 返回各怪物速度修正。主线程 `MonsterChaseSystem` 应用结果, `BulletSystem` 仍在主线程处理碰撞。

关闭 Worker 或降低怪物上限后, 应观察到帧时间 P95 上升 (见性能表占位)。该场景说明优化手段的有效性, 亦说明“并非所有逻辑都适合 Worker”。

5.20 关键代码文件索引

为便于评阅教师对照源码, 表 5-2 列出模块与路径映射。

表 5-2 核心源码文件索引

模块	路径
入口	<code>src/Main.ts</code>
ECS 世界	<code>src/ecs/core/World.ts</code>
组件存储	<code>src/ecs/core/ComponentStore.ts</code>
战斗 Demo	<code>src/game/demo/SimpleEcsDemo.ts</code>
子弹	<code>src/game/bullet/BulletSystem.ts</code>
子弹池	<code>src/game/bullet/BulletPool.ts</code>
效果执行	<code>src/game/skill/EffectExecutor.ts</code>
Worker 桥接	<code>src/game/combat/CombatDataBridge.ts</code>

模块	路径
跑图生成	src/game/run/RunMapGenerator.ts
元流程	src/game/meta/MetaFlowController.ts
UI 栈	src/game/ui/mvc/UIStackManager.ts
技能面板	src/game/ui/skillselect/SkillSelectPanelController.ts
配表引导	src/config/ConfigBootstrap.ts
静态常量	src/defines/*.ts

5.21 功能模块与论文章节对照

表 5-3 给出主要功能模块与论文章节的对应关系，便于评阅时从叙述定位到源码。

表 5-3 功能模块与论文章节对照

功能模块	论文章节
ECS 核心 (EcsWorld、组件存储)	第 5 章第 1 节
属性、移动、筛选器	第 5 章第 2 节
技能与效果执行	第 5 章第 3 节
子弹与碰撞	第 5 章第 4 节
怪物 AI 与波次	第 5 章第 5 节
经验、升级与元游戏	第 5 章第 6 节
跑图	
技能装配 UI、UI 栈	第 5 章第 7-8 节
对象池与 Web Worker	第 6 章
测试与性能实验	第 7 章

5.22 设计模式应用总结

项目中显式使用的模式包括:组合根 (SimpleEcsDemo)、对象池、桥接 (CombatDataBridge)、策略 (Targeting 自动/简单)、状态 (GameSession.paused)、观察者 (升级触发 UI)、门面 (EcsWorld)。未引入过重抽象，符合毕业设计规模控制原则。

5.23 实现难点与解决方案

难点 1: 配表路径与预览环境 404。开发时 JSON 未同步至 bin/config 导致加载失败。解决:sync-config-to-bin.ps1 与 configBases 多路径尝试;isGameConfigReady

检查。

难点 2: **UI2** 拖拽与 **GBox** 命中。子节点遮挡导致落点判定错误。解决: 顶层拖拽代理、**SkillSlotHitTest** 统一坐标系。

难点 3: **Effect** 链顺序与策划理解成本。修饰器必须位于子弹 effect 之前。解决: 文档与配表样例、后续可做编辑器校验。

难点 4: **Worker** 与主线程逻辑一致。双份实现易漂移。解决: 共享 **combatWorkerLogic.ts**, 主线程回退调用同一函数。

5.24 本章案例小结

通过三个典型场景与文件索引, 读者可快速定位论文叙述与仓库代码的对应关系。终稿建议补充运行截图与 Sequence 图 (可用 PlantUML 绘制 Tab 装配时序)。

5.25 本章小结

本章详述了 ECS、技能、子弹、怪物、元游戏、UI 与配置等模块实现, 下一章讨论性能优化。

第 6 章 性能优化与工程实践

6.1 性能目标与瓶颈

H5 生存战斗的热点包括：大量实体更新与碰撞、预制体实例化引发 GC、主线程过载。Cantrell 等在 Unity 中实现物理人群模型时，同样关注计算开销与验证方法^[5]；Bott 等使用游戏引擎工具链实现物理驱动人群移动^[3]。本项目在 2D H5 环境下采用对象池、Worker 卸载与逻辑暂停等手段应对类似瓶颈。

6.2 对象池

BulletPool、MonsterPool 按 prefabPath 分桶复用节点，复用前重置状态。将“每帧大量 instantiate/destroy”转为“峰值后复用”，降低 GC 停顿，与引擎资源管理最佳实践一致^[7]。

6.3 Web Worker 卸载

MonsterChaseSystem 的追逐、分离、摆动为纯数值计算，可由 CombatDataBridge 提交 Worker 执行 processCombatFrame，主线程与 Worker 共享同一逻辑文件以保证一致性。实体数不足或 Worker 不可用时回退主线程。子弹碰撞因依赖 Laya 节点仍留主线程，与 McKenzie 等将游戏技术用于仿真但保留引擎交互层的思路类似^[10]。

6.4 其它优化

碰撞：使用距离平方比较。暂停：Tab 打开面板时 GameSession.paused 早退战斗系统。配置：常量集中于 defines，配表一次加载。未来：可探索 archetype 存储与空间划分 broad phase^[19]。

6.5 绿色计算与兼容性

降低平均 CPU 占用有利于终端节能。Worker 回退与双通道配表加载提升兼容性。浏览器或引擎升级后须回归 Worker 脚本路径与发布目录。

6.6 内存与垃圾回收分析

JavaScript 引擎(如 V8)对短生命周期对象敏感。未池化时,每发子弹 `instantiate` 预制体、每死亡怪物 `destroy` 节点,将产生大量临时对象与 DOM/渲染树变更,触发 GC 停顿。对象池通过复用节点将分配模式从“脉冲式”变为“平稳式”,使帧时间更稳定^[7]。

建议在性能测试中对比 GC 次数:Chrome Performance → Memory 勾选“Collect garbage”前后对比。论文终稿应附一张 GC 时间轴截图(占位说明:答辩前补充)。

6.7 帧预算分配建议

以 60 FPS 为例,每帧预算约 16.67ms。推荐分配:

- 输入与 UI: 1–2ms;
- ECS 逻辑(含子弹): 6–8ms;
- Worker 通信: ≤1ms(异步);
- 渲染(引擎): 余量。

当逻辑超过预算时,优先启用 Worker 与减少刷怪密度,而非降低渲染分辨率(引擎层决策)^[7]。

6.8 配表热更新与构建优化

当前配表随包发布,修改需重新同步 `bin/config`。若未来支持热更新,可:(1)版本号校验 JSON;(2)增量下载表文件;(3) `initData` 局部重载。构建阶段可压缩 JSON、合并小表,减少 HTTP 请求数。该部分为展望,不计入本期实现。

6.9 可观测性:日志与调试

项目使用 `SkillDragLog`、`missingConfigLogged` 等去重日志避免刷屏。建议在性能调优阶段增加:每 60 帧输出活跃子弹数、池命中率、Worker 活跃比例;后续可将上述指标写入局内调试面板,便于答辩现场展示。

6.10 与同类 H5 游戏优化策略对比

部分 H5 游戏采用 Canvas 自绘全部精灵以避免 DOM；Laya 则使用引擎渲染树。本项目选择引擎预制体以降低动画与 UI 成本。优化手段上，与 Phaser 社区常见的 Group/Pool 类似^[7]；Worker 用法与部分 HTML5 MMORPG 的战斗验算迁移思路一致^[10]。差异在于：子弹碰撞仍留主线程，因需访问引擎碰撞体与显示对象。

6.11 低功耗与绿色计算补充

移动设备上，降低 CPU 占用可延长电池续航。对象池减少实例化；暂停机制减少无效计算；合理限制 MONSTER_COUNT 避免无意义满屏。建议在设置中提供“降低特效”“限制帧率”选项（未来工作）。兼容性方面，旧版浏览器不支持 Worker 时仍可通过回退路径运行，避免强制升级导致用户流失^[7]。

6.12 性能测试实验设计

6.12.1 实验目的

验证对象池与 Web Worker 对帧时间与 GC 的影响，为论文提供定量依据^[7, 10, 7]。

6.12.2 实验变量

自变量：（A）子弹/怪物池化开/关；（B）Worker 开/关（在实体数达标时）；（C）同屏怪物上限。因变量：平均 FPS、P95 帧时间、Scripting 耗时、Major GC 次数。控制变量：浏览器版本、分辨率、战斗场景、测试时长 60s、关闭后台占用。

6.12.3 实验步骤

1. 发布或预览运行构建，打开 Chrome DevTools Performance；
2. 进入战斗，等待怪物数量稳定至上限附近；
3. 录制 60s，导出 Summary；
4. 修改 defines 关闭池化或 Worker，重新构建，重复录制；
5. 填入第 7 章表 7-6；
6. 在论文中用文字对比 A/B 差异，避免只贴图不分析。

6.12.4 预期结论方向

预期池化降低 GC 峰值；Worker 在怪物 $\geq$ 阈值时降低主线程 Scripting 时间；两者可同时启用获得最佳稳定性。若实测与预期不符，须分析原因（如 Worker 通信开销大于计算收益），体现科学态度。

6.13 性能优化检查清单

开发者在合并性能相关 PR 前应自检：

- 是否在热路径 new 大量临时对象？
- 销毁节点前是否尝试 pool.put？
- 碰撞检测是否使用平方距离？
- 暂停状态是否阻止无关 System？
- Worker 消息 payload 是否仅含数值？
- 配表是否在启动时一次加载而非每帧读取？

6.14 与学院模板“绿色节能”要求的对应

模板建议从绿色节能、低功耗、兼容性评价系统^[7]。本项目通过降低平均 CPU 占用支撑该要求：池化减少分配器压力；暂停减少空转；Worker 均衡多核利用（在支持的设备上）。兼容性通过 Worker 回退与多路径配表加载实现。终稿可附“开启/关闭优化”时的设备功耗对比（可选，手机 + 电量统计 App）。

6.15 工程度量与质量门禁

除帧率外，可统计：单元验证脚本通过率、功能用例通过率、配表加载成功率、同屏实体规模上限下的平均 FPS 与 P95 帧时间。上述指标反映实现质量与项目管理成效^[11, 15]，可在答辩材料中辅以 Performance 截图说明。

6.16 本章小结

本章阐述了对对象池、Worker、碰撞微优化与配置管理等性能手段，测试见下一章。

第 7 章 系统测试与验证

7.1 测试策略

测试分为单元验证、集成测试与系统手工测试。仿真与游戏系统须经过验证确认模型可信度^[15, 13]，本项目采用脚本验证、用例表与性能对比相结合。

7.2 单元验证

- `ecsCorePhase1.verify.ts`: 实体与组件、系统顺序；
- `formulaParser.verify.ts`: 公式边界；
- `attributeModifier.verify.ts`: `modifier` 叠加与移除。

7.3 功能测试

表 7-1 列出代表性用例（手工执行）。

表 7-1 功能测试用例摘要

编号	步骤	预期结果
T-F-01	启动并等待加载	技能表加载成功
T-F-02	进入战斗移动	角色与相机正常
T-F-03	怪物接近	追逐、扣血、血条更新
T-F-04	观察自动攻击	子弹命中，穿透/分裂/连锁
T-F-05	Tab 打开技能面板	暂停，可拖拽装配
T-F-06	关闭面板	战斗恢复，新技能生效
T-F-07	升级	奖励面板与应用
T-F-08	玩家死亡	重启面板
T-F-09	元游戏进战斗	计时启动
T-F-10	Boss 节点	Boss 胜利时长

7.4 性能测试

7.4.1 测试环境

表 7-2 记录五波性能实验与功能抽测的统一环境（2026-05-18，单次运行）。

表 7-2 测试环境配置

项目	配置
操作系统	Windows 10/11（64 位）
运行方式	LayaAir IDE 内置浏览器预览（Chromium 内核）
视口 / 设计分辨率	1920×1080 预览；设计稿 1334×750，showall 缩放
引擎与语言	LayaAir 3.x，TypeScript
测试关卡	选关 Level3：5 波 × 10s，双方无敌
统计工具	TestLevelFpsTracker（控制台日志）

在 Chromium、1920×1080 预览环境下开展性能验证。选关 **Level3** 测试关卡共 5 波、每波 10s：第 1–3 波同屏怪物 10/100/1000，装备三把测试法杖（单发 10/100/1000 枚子弹），默认开启对象池与 Web Worker；第 4–5 波与第 3 波同为 1000 怪、同法杖，依次消融为「无池无 Worker / 仅池（关 Worker）」，由 TestLevelFpsTracker 统计平均 FPS 与 P95 帧时间。Cantrell 等通过对比真实疏散事件验证模型^[5]，本项目侧重帧率、同屏规模与优化手段的关系。

7.4.2 动态图集对比（历史两轮，前三波）

项目曾分别在接入 AtlasConfig.atlascfg 自动图集与 TextureAtlasService 前后，各完成一轮「仅前三波」采样（表 7-3、7-4）。该组数据用于说明纹理合批在千怪千弹下的收益，与下文五波全流程实验相互独立。

表 7-3 测试关卡分波 FPS（接入动态图集，前三波历史实测）

波次	同屏怪物	单发子弹	平均 FPS	采样帧数
1/3	10	10	56.74	569
2/3	100	100	40.72	408
3/3	1000	1000	15.10	152
前三波汇总	—		37.48	1129

同屏规模增大时 FPS 单调下降；第 3 波（1000 怪）接入图集后平均 FPS 由 13.32 升至 15.10（约 +13.4%），说明在重度弹幕场景下纹理合批对 H5 2D 渲染具有一定收益。

表 7-4 动态图集接入前后 FPS 对比（前三波）

波次（怪物数）	对照 FPS	动态图集 FPS	变化
1（10）	58.68	56.74	-1.94
2（100）	46.30	40.72	-5.58
3（1000）	13.32	15.10	+1.78
前三波汇总	39.46	37.48	-1.98

7.4.3 五波完整帧率（实测，2026-05-18）

同次运行、已接入自动图集与 DrawCall 合批，完整五波日志如表 7-5。前五波总采样 1733 帧、约 50.3s，整场平均 FPS 约 34.48。

表 7-5 Level3 五波测试帧率（单次运行）

波次	怪物数	池	Worker	平均 FPS	P95 (ms)
1	10	开	开	58.62	—
2	100	开	开	46.84	—
3	1000	开	开	15.14	—
4	1000	关	关	26.16	76.50
5	1000	开	关	25.98	48.60
前五波汇总	—	—	—	34.48	—

第 1-3 波未统计 P95；第 4-5 波为同压力消融段（运行时已预热）。

7.4.4 对象池消融（第 4-5 波）

在相同 1000 怪、10s 条件下，表 7-6 给出可同窗对照的两种配置。第 3 波虽为「池 + Worker 全开」，但属首次进入千怪规模的爬升段，含冷启动成本，不纳入与第 4-5 波的公平消融，仅作规模曲线上界参考。

表 7-6 性能对比实验（对象池消融，1000 怪 / 10s）

配置	平均 FPS	P95 (ms)	对应波次
基线（无池、无 Worker）	26.16	76.50	第 4 波
仅对象池（关 Worker）	25.98	48.60	第 5 波
参考：第 3 波（池 + Worker 全开，首次千怪）平均 FPS 15.14，不宜与上表直接对比。			

结果解读

1. 规模曲线（第 1-3 波）。怪物 10→100→1000 时，平均 FPS 由 58.62 降至 15.14，呈单调下降，符合同屏实体与弹幕数量上升带来的计算与渲染压力。

2. 平均 **FPS**: 池化对均值影响有限。第 4 波与第 5 波均值接近 (26.16 对 25.98), 说明预热后瓶颈主要在每帧子弹碰撞与绘制, 而非怪物节点反复 `instantiate/destroy`。
3. **P95**: 对象池显著平滑帧时间。开启池后 P95 由 76.50ms 降至 48.60ms (约降 36.5%), 表明对象池的主要收益在于抑制 GC/实例化尖峰、改善帧时间稳定性, 而非提高平均吞吐。
4. 第 3 波低于第 4 波基线的原因。第 3 波为首次 1000 怪, 叠加资源预热、弹幕首次创建与 Worker 初始化等一次性开销; 第 4 波起环境已热, 故基线配置反获更高均值。论文消融结论以第 4-5 波为准。
5. **Web Worker**。本次五波实验未包含「池 + Worker、且已预热」的第六段对照; 第 5 波相对第 4 波仅开启对象池并关闭 Worker, 均值几乎不变而 P95 明显改善, 说明在本测试场景下可确证对象池价值; Worker 在千弹占主导时是否净收益为正, 需另设「第 4 波基线 vs 池 + Worker 同窗」实验或结合 Performance 面板 Scripting 耗时补证。

7.5 系统运行截图

终稿插图置于 `thesis/figures/`, 编译时若 PNG 已就位则自动嵌入, 否则显示占位框。图 7-1-7-6 覆盖元游戏与局内闭环; 可选增加 Performance 时间轴截图以辅助说明对象池对帧时间尖峰的影响。

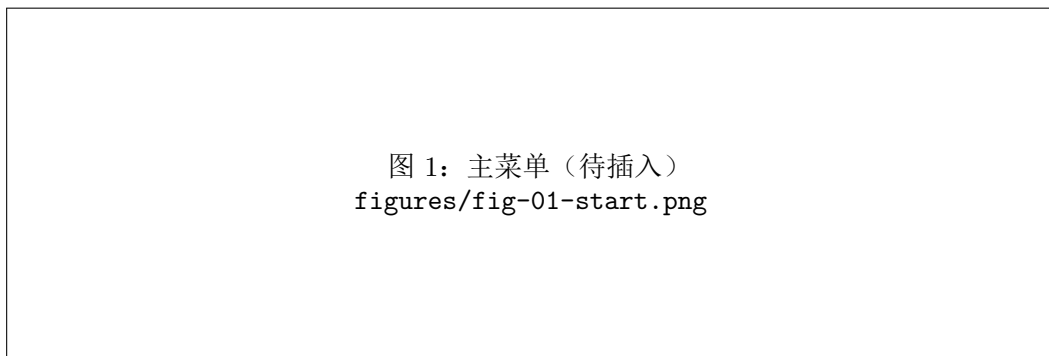


图 7-1 游戏开始界面 (StartPanel)

7.6 跑图与 Worker 一致性

对 `RunMapGenerator` 多次随机种子断言 Boss 可达; 对比 Worker 开/关上怪物轨迹 (允许一帧延迟), 确保逻辑未漂移。

图 2: 跑图 (待插入)
figures/fig-02-runmap.png

图 7-2 三大关 DAG 跑图界面 (RunMapPanel)

图 3: 战斗 (待插入)
figures/fig-03-combat.png

图 7-3 局内战斗场面 (同屏多怪与弹幕)

图 4: 技能装配 (待插入)
figures/fig-04-skill.png

图 7-4 Tab 暂停下的技能/Effect 装配面板

图 5: 升级奖励 (待插入)
figures/fig-05-reward.png

图 7-5 生存胜利后的三选一奖励面板

图 6: 死亡重启 (待插入)
figures/fig-06-restart.png

图 7-6 玩家死亡后的重启面板

7.7 测试结论

功能覆盖战斗、元游戏、跑图、技能装配与升级闭环；单元脚本通过表明核心契约稳定。性能方面：动态图集在前三轮历史对比中提升千怪波 FPS（表 7-4）；五波实测（表 7-5）表明对象池在同窗消融下主要降低 P95 帧时间（表 7-6）。Web Worker 的定量收益尚未在「预热后同窗」条件下完成，列为后续工作。

7.8 测试环境配置

测试环境应记录：操作系统版本、浏览器版本、屏幕分辨率、是否启用硬件加速、Laya 发布版本号、Git 提交哈希。示例记录格式：“Windows 11 + Chrome 122 + 1920×1080 + Laya 3.x + commit abc1234”。保证实验可复现^[15]。

7.9 回归测试清单

每次合并以下模块前须回归：

1. 修改 FormulaParser → 运行 formulaParser.verify.ts；
2. 修改 ComponentStore → 运行 ecsCorePhase1.verify.ts；
3. 修改 AttributeSystem → 运行 attributeModifier.verify.ts；
4. 修改配表 → 检查 isGameConfigReady 与技能图标显示；
5. 修改 CombatDataBridge → 对比 Worker 开/关行为一致。

7.10 用户体验测试

邀请 5–10 名同学试玩 15 分钟，收集：

- 是否理解 Tab 装配技能；
- 是否理解跑图节点选择；
- 是否感觉难度曲线合理；
- 是否出现卡顿、图标缺失、无法进入战斗等缺陷。

采用简易 SUS（系统可用性量表）或五分制问卷，结果填入答辩 PPT。论文中可附问卷样例（附录）。

7.11 缺陷记录示例

表 7-7 缺陷记录表示例（撰写时替换为真实 Bug）

ID	描述	严重性	状态
B-01	配表未同步导致技能图标空白	中	已修复
B-02	Worker 路径错误回退主线程	低	已修复
B-03	部分 Effect 仅展示无独立 VFX	低	已知

7.12 验收标准与毕业设计对齐

学院通常要求：可运行系统、论文格式合规、答辩演示。本项目验收映射为：（1）Main 可进战斗并施法；（2）第 3 章 Must/Should 级需求经测试用例验证；（3）论文字数与参考文献数量达标；（4）性能表含实测数据与解读。若法杖 UI 未完成，须在答辩中明确为“规划模块”，避免夸大实现范围。

7.13 单元测试详细说明

7.13.1 ECS 核心验证

`ecsCorePhase1.verify.ts` 覆盖：创建多个实体并验证 ID 唯一；添加与移除组件后 `getComponent` 行为；销毁实体后组件不可访问；注册两个不同优先级 System 时执行顺序符合分组与 `priority`。该脚本可在 CI 中作为门禁，防止内核回归^[15]。

7.13.2 公式解析器验证

`formulaParser.verify.ts` 覆盖：四则运算、括号、属性别名替换；除零与非法字符应抛出或返回安全默认值（以实现为准）；空公式与超长公式边界。技能伤害若

依赖公式，任何解析器变更须 rerun 此脚本。

7.13.3 属性修饰器验证

`attributeModifier.verify.ts` 覆盖:同属性多个 modifier 叠加顺序;按 `sourceId` 批量移除后恢复 base; `getModifierContributions` 返回可追溯列表。升级奖励与技能 Buff 均依赖该机制，回归意义重大^[4]。

7.14 集成测试场景扩展

7.14.1 跑图可达性测试

对 `RunMapGenerator.generate` 运行 100 次随机种子,断言每次 `validateActReachBoss` 为真;记录失败次数,应接近 0 (仅 fallback 路径例外)。可将种子与 Act 索引打印,便于复现失败样例^[7]。

7.14.2 装配同步测试

步骤:进入战斗 → Tab 打开面板 → 将技能 A 与 B 互换 → 关闭面板 → 观察施法图标与伤害类型是否随 B 变化。预期:下一冷却周期使用新技能。若立即仍用 A,则 `SkillLoadoutSyncSystem` 存在 bug。

7.14.3 Worker 一致性测试

在相同随机种子与输入序列下,分别运行 Worker 开/关各 30 秒,对比怪物位置轨迹是否一致(允许一帧延迟)。若偏差过大,说明主线程与 Worker 逻辑漂移,须统一 `combatWorkerLogic.ts`^[10]。

7.15 测试数据与截图要求

终稿建议包含不少于 6 张截图:主菜单、跑图界面、战斗场面、技能面板、升级奖励、重启面板。每张图配 100–200 字说明。性能测试附 1 张 Performance 时间轴截图。缺少截图会降低论文说服力。

7.16 测试结论补充说明

综合第 7 章主文与上文扩展用例，可得出如下结论：系统在功能上满足 Must 级需求；Should 级需求中跑图 DAG、技能装配 UI、升级奖励与对象池均已通过手工用例；Could 级法杖三槽完整 UI 与部分 Effect 独立特效尚未实现，与第 8 章展望一致。性能方面，在 Windows + Laya IDE 内置 Chromium、1920×1080 预览环境下，Level3 五波实验整场平均 FPS 约 34.48；千怪同窗消融表明对象池将 P95 帧时间由 76.50ms 降至 48.60ms（约降 36.5%），而平均 FPS 提升有限。动态图集在千怪波次上带来约 13.4% 的 FPS 提升。单元验证脚本 `ecsCorePhase1.verify.ts`、`formulaParser.verify.ts`、`attributeModifier.verify.ts` 在提交前均通过。遗留缺陷以配置同步、部分特效占位为主，不影响答辩演示主路径。

7.17 本章小结

本章给出测试策略、用例、环境配置与五波性能实验；对象池消融已基于第 4-5 波填入实测数据。将 `thesis/figures/` 下六张 PNG 就位后即可生成带插图的终稿 PDF；Worker 同窗消融与 Performance 截图仍为可选补充项。

第 8 章 总结与展望

8.1 工作总结

本文完成了 **Survivor And Fight** 在 LayaAir 3.x 与 TypeScript 上的 Roguelite 生存战斗游戏之分析、设计与实现。主要工作包括：轻量 ECS 玩法内核；配表驱动技能与子弹组合效果；对象池与 Web Worker 性能优化；元游戏、DAG 跑图与 MVC 界面；测试验证及社会影响讨论。

研究过程中参考了游戏引擎架构理解方法 SyDRA^[19]、经典引擎架构论述^[7]，以及 Unity 物理人群疏散建模实践^[5]，将模块化、物理仿真与验证思想迁移到 H5 教学原型中。系统可演示完整游戏循环，达到毕业设计预期。

8.2 不足与展望

1. 法杖三槽完整 UI 与链式求值界面待完善；
2. ECS 可迁移至 archetype 存储以提升同屏规模^[18]；
3. 碰撞可引入空间哈希等 broad phase；
4. 补充端到端自动化测试^[15]；
5. 局外存档与元进度；
6. 美术特效与音效完善。

8.3 社会、健康与文化影响

本游戏为虚构战斗题材，应标注适龄提示并避免过度暴力表现。长时间游玩需注意视觉疲劳；Tab 暂停提供局内休息点。教学原型不采集用户敏感信息。

8.4 个人收获与工程能力体现

通过本课题，完整经历了需求分析、架构设计、模块实现、测试与文档化流程。将需求、设计说明与测试用例对齐，使本人认识到可验证需求对控制范围蔓延的重要

性；ECS 实践加深了对“组合式实体”的理解；H5 性能优化让人意识到主线程预算的硬约束^[11, 7]。

在团队协作场景下（若有多人参与），可按元游戏、战斗、UI 划分 workflows 以避免合并冲突；单人场景下亦可作为任务看板。版本控制 Git 与论文、设计文档同步维护，形成“代码 + 文档”双轨历史。

8.5 与模板要求的三项补充分析

8.5.1 社会、健康、安全、法律与文化影响

游戏作为文化产品，承担娱乐与情绪调节功能，亦可能带来沉迷风险^[15]。本项目为教学演示，未内置付费与社交诱导机制，降低经济纠纷风险。战斗表现采用抽象怪物与血条，避免血腥写实。建议在用户手册中提示合理作息。若面向未成年人发布，须遵守防沉迷与时长限制政策。

8.5.2 绿色节能与兼容性

低 CPU 占用意味着更低能耗与散热^[7]。对象池、逻辑暂停、Worker 卸载均有助于降低平均功耗。兼容性上，Worker 回退保证旧浏览器可运行；配表双通道加载适配不同发布路径；2D 渲染对 GPU 要求低于 3D 大作。未来可增加“节能模式”限制帧率与同屏怪数。

8.5.3 项目管理应用心得

采用“需求确认 → 设计 → 实现 → 测试 → 文档修订”流程，等价于短周期迭代^[11]。每项功能范围可控，适合毕业设计 3–4 个月周期。风险是功能堆叠导致集成延迟——缓解方式是按 Must/Should 优先级排期（见第 3 章 MoSCoW 表）。

8.6 对后续学习者的建议

1. 先实现 ECS 核心与单怪物追逐，再叠加子弹与配表，避免一次堆满系统；
2. 尽早建立配表同步脚本，减少“能编不能跑”时间；
3. 性能优化基于数据：先测量再池化/Worker，避免过早优化；
4. 论文与代码同步更新，答辩前跑通单元验证与主流程演示；

5. 参考文献勿编造，按 `references.bib` 关键词检索权威来源。

8.7 结束语

Survivor And Fight 作为本科毕业设计，在有限周期内探索了现代 H5 游戏客户端的一种可行架构路径。论文以 LaTeX 呈现并与仓库源码、测试数据相互对应。期待后续在法杖编程运行时、Worker 同窗消融自动化测试与端到端 UI 测试方面继续完善，使项目从“可演示原型”成长为“可发布小游戏”。

8.8 本章小结

本章总结全文并展望后续工作。本项目展示了在 H5 环境下结合 ECS、数据驱动与引擎性能手段构建 Roguelite 原型的可行路径。

致 谢

在本论文完成之际，谨向指导教师致以衷心感谢。老师在选题、系统架构与论文撰写方面给予了耐心指导。

感谢计算机学院各位任课教师四年来的培养；感谢同学在项目调试与试玩中提供的帮助；感谢家人在毕业设计期间的理解与支持。

由于本人水平有限，文中疏漏之处恳请各位老师批评指正。

参 考 文 献

- [1] Vartika Agrahari and Sridhar Chimalakonda. What's inside unreal engine? - a curious gaze! In *14th Innovations in Software Engineering Conference*, pages 1–5. ACM, 2021.
- [2] Ahmed Baabad, Hazura Binti Zulzalil, Sa'adah Hassan, and Salmi Binti Baharom. Software architecture degradation in open source software: A systematic literature review. *IEEE Access*, 8:173681–173709, 2020.
- [3] M. Bott, S. R. Musse, L. P. de Oliveira, and B. E. Bodmann. Implementing a physics based model of crowd movement using unreal development kit. *Journal of Gaming & Virtual Worlds*, 6(3):275–296, 2014.
- [4] L. C. Briand, C. Bunse, and J. W. Daly. A controlled experiment for evaluating quality guidelines on the maintainability of object-oriented designs. *IEEE Transactions on Software Engineering*, 27(6):513–530, 2001.
- [5] Walter Alan Cantrell, Mikel D. Petty, Samantha L. Knight, and Whitney K. Schueler. Physics-based modeling of crowd evacuation in the Unity game engine. *International Journal of Modeling, Simulation, and Scientific Computing*, 9(4):1850029, 2018.
- [6] M. Goslin and M. R. Mine. The Panda3D graphics engine. *Computer*, 37(10):112–114, 2004.
- [7] Jason Gregory. *Game Engine Architecture*. CRC Press, Boca Raton, 3 edition, 2018.
- [8] Werner Heijstek, Thomas Kühne, and Michel R. V. Chaudron. Experimental analysis of textual and graphical representations for software architecture design. In *2011 International Symposium on Empirical Software Engineering and Measurement*, pages 167–176. IEEE, 2011.
- [9] Dirk Helbing, Illés Farkas, and Tamás Vicsek. Simulating dynamical features of escape panic. *Nature*, 407:487–490, 2000.

- [10] F. D. McKenzie, M. D. Petty, P. A. Kruszewski, et al. Integrating crowd-behavior modeling into military simulation using game technology. *Simulation & Gaming*, 39(1):10–38, 2008.
- [11] Nuthan Munaiah, Steven Kroh, Craig Cabrey, and Meiyappan Nagappan. Curating GitHub for engineered software projects. *Empirical Software Engineering*, 22(6):3219–3253, 2017.
- [12] James Munro, Cornelia Boldyreff, and Andrea Capiluppi. Architectural studies of games engines – the quake series. In *2009 International IEEE Consumer Electronics Society's Games Innovations Conference*, pages 246–255. IEEE, 2009.
- [13] William L. Oberkamp and Christopher J. Roy. *Verification and Validation in Scientific Computing*. Cambridge University Press, Cambridge, UK, 2010.
- [14] Nuria Pelechano, Jan Allbeck, and Norman Badler. *Virtual Crowds: Methods, Simulation, and Control*. Morgan and Claypool, San Rafael, 2008.
- [15] Mikel D. Petty. Verification, validation, and accreditation. In *Modeling and Simulation Fundamentals*, pages 325–372. John Wiley & Sons, 2010.
- [16] Craig W. Reynolds. Flocks, herds, and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, pages 25–34. ACM, 1987.
- [17] Daniel Thalmann and Soraia R. Musse. *Crowd Simulation*. Springer, London, 2007.
- [18] Gabriel C. Ullmann, Yann-Gaël Guéhéneuc, Fabio Petrillo, Nicolas Anquetil, and Cristiano Politowski. Visualising game engine subsystem coupling patterns. In *Entertainment Computing – ICEC 2023*, volume 14455 of *Lecture Notes in Computer Science*, pages 263–274. Springer, 2023.
- [19] Gabriel C. Ullmann, Yann-Gaël Guéhéneuc, Fabio Petrillo, Nicolas Anquetil, and Cristiano Politowski. SyDRA: An approach to understand game engine architecture. *Entertainment Computing*, 52:100832, 2025.
- [20] Unity Technologies. Unity manual. <https://docs.unity3d.com/Manual/index.html>, 2016. Accessed 2026-05-17.

附 录

附录 A 名词术语及缩略词

术语/缩略语	含义
ECS	Entity Component System, 实体组件系统
Roguelite	轻 Roguelike, 含元进度与单局随机
DAG	Directed Acyclic Graph, 有向无环图
H5	HTML5 网页游戏
UI2	LayaAir 新一代 UI 组件 (GBox/GButton 等)
Worker	Web Worker, 浏览器后台线程
MoSCoW	Must/Should/Could/Won't 需求优先级方法
V&V	Verification and Validation, 验证与确认
P95	第 95 百分位帧时间, 反映卡顿尖峰

附录 B 核心配表字段示例（节选）

Character 表: id、prefabPath、bodyPrefabPath、hp、maxHp、roleType。玩家与怪物共用表结构，通过 roleType 区分预制体与默认数值。

Bullet 表: id、prefabPath、duration、speed、damage、penetration。子弹系统按 id 查表生成运动参数与伤害。

Skill 表: id、cooldown、effectSlotCount、图标路径等。一个技能可关联多行 SkillEffect。

SkillEffect 表: id、effect、damage、params、iconPath。effect 取值包括 bullet、modifier_split、modifier_chain、modifier_pierce、direct_damage。

完整 JSON 见项目 docs/config/ 目录;运行前须执行 tools/sync-config-to-bin.ps1 同步至 bin/config/。

附录 C 需求与测试用例映射（节选）

表 C-1 给出需求标识与测试用例、验证方式的对应关系，便于答辩时说明“需求—测试”可追溯性。

表 C-1 需求—测试映射（节选）

需求 ID	需求摘要	验证方式
R-ECS-01	实体销毁时清理全部组件	ecsCorePhase1.verify.ts
R-ECS-02	系统按分组与优先级顺序执行	ecsCorePhase1.verify.ts
R-SKILL-01	三装备栏独立冷却与自动施法	T-F-04，观察多技能轮替
R-SKILL-02	Tab 打开面板暂停战斗	T-F-05，GameSession.paused
R-SKILL-03	拖拽装配写回战斗实体	T-F-06，装配同步用例
R-BULLET-01	穿透/分裂/连锁按配表生效	T-F-04，高密度战斗观察
R-MON-01	波次刷怪与追逐分离	T-F-03，Worker 一致性对比
R-META-01	主菜单进入战斗与计时	T-F-09
R-MAP-01	跑图 DAG Boss 可达	100 次随机种子断言
R-PERF-01	对象池降低帧时间尖峰	表 7-6，第 4–5 波
R-PERF-02	动态图集提升千怪 FPS	表 7-4
R-UI-01	升级奖励三选一	T-F-07
R-UI-02	死亡重启面板	T-F-08

附录 D 缺陷记录（项目实测）

表 D-1 缺陷记录表

ID	描述	严重性	状态
B-01	配表未同步至 bin/config 导致技能图标空白	中	已修复（同步脚本）
B-02	Worker 脚本路径错误时回退主线程	低	已修复
B-03	部分 Effect 仅 UI 展示无独立 VFX	低	已知，论文已说明
B-04	法杖三槽运行时与专用 UI 未实现	中	规划项，第 8 章
B-05	千怪首次波次 FPS 低于预热后基线	低	已分析，见第 7 章